Natalie Leung
014790932
18 October 2026

Data 260 Homework 6

Personal Configuration and Domain

SID4	0932
PORT_BASE	8032
PREFIX	s0932
SEED	0932
VERIFY_SEED	260932
DOMAIN_ID	4 (open-source package vulnerabilities)

Hardware: nvdia geforce 4070, msi cpu
Local model used: ollama qwen3:8b
Tagged commit hash repo with most updated codes (submission on canvas only):
https://github.com/n-k-leung/data260-0932

Part 1 - MongoDB Document API

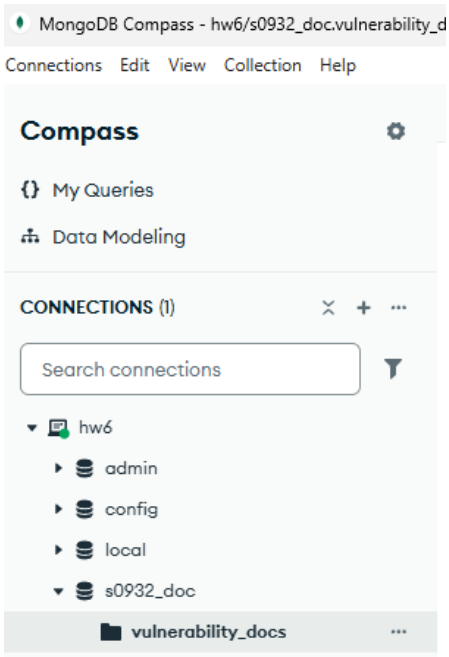
I. Database
• Create the MongoDB database <PREFIX>_doc with a collection for a document-oriented representation of your domain (e.g., for a task-management-style setup this would be a "tasks" collection - map the fields below to your own domain entity instead).

reports > hw06 > backend > .env

```
DATABASE_URL=mysql+pymysql://root:seekrit@localhost:3306/s0932_rel
MONGODB_URI=mongodb://localhost:27017
MONGO_DB=s0932_doc
```

```
MONGODB_URI = os.getenv("MONGODB_URI", "mongodb://localhost:27017")
MONGODB_DB = os.getenv("MONGODB_DB", PREFIX + "_doc")
COLLECTION_NAME = "vulnerability_docs"

MESSAGES_COLLECTION = "messages"
SUMMARIES_COLLECTION = "summaries"
EPISODES_COLLECTION = "episodes"
client = MongoClient(MONGODB_URI)
```



• Include at least: one required short text field (max 100 characters), one optional description field, two constrained categorical fields (e.g., a status-style enum with a default, and a category-style field with a fixed set of options), one required date/time field, and automatic created/updated timestamps.

```python
StatusType = Literal["open", "triaged", "fixed", "wont_fix"]
CategoryType = Literal["injection", "xss", "auth", "dos", "other"]

class VulDocCreate(BaseModel):
    title: str = Field(min_length=1, max_length=100)
    description: Optional[str] = None
    status: StatusType = "open"
    category: CategoryType
    discovered_at: datetime

class VulDocUpdate(BaseModel):
    title: Optional[str] = Field(default=None, min_length=1, max_length=100)
    description: Optional[str] = None
    status: Optional[StatusType] = None
    category: Optional[CategoryType] = None
    discovered_at: Optional[datetime] = None

class VulDocOut(BaseModel):
    id: str
    title: str
    description: Optional[str] = None
    status: StatusType
    category: CategoryType
    discovered_at: datetime
    created_at: datetime
    updated_at: datetime
```

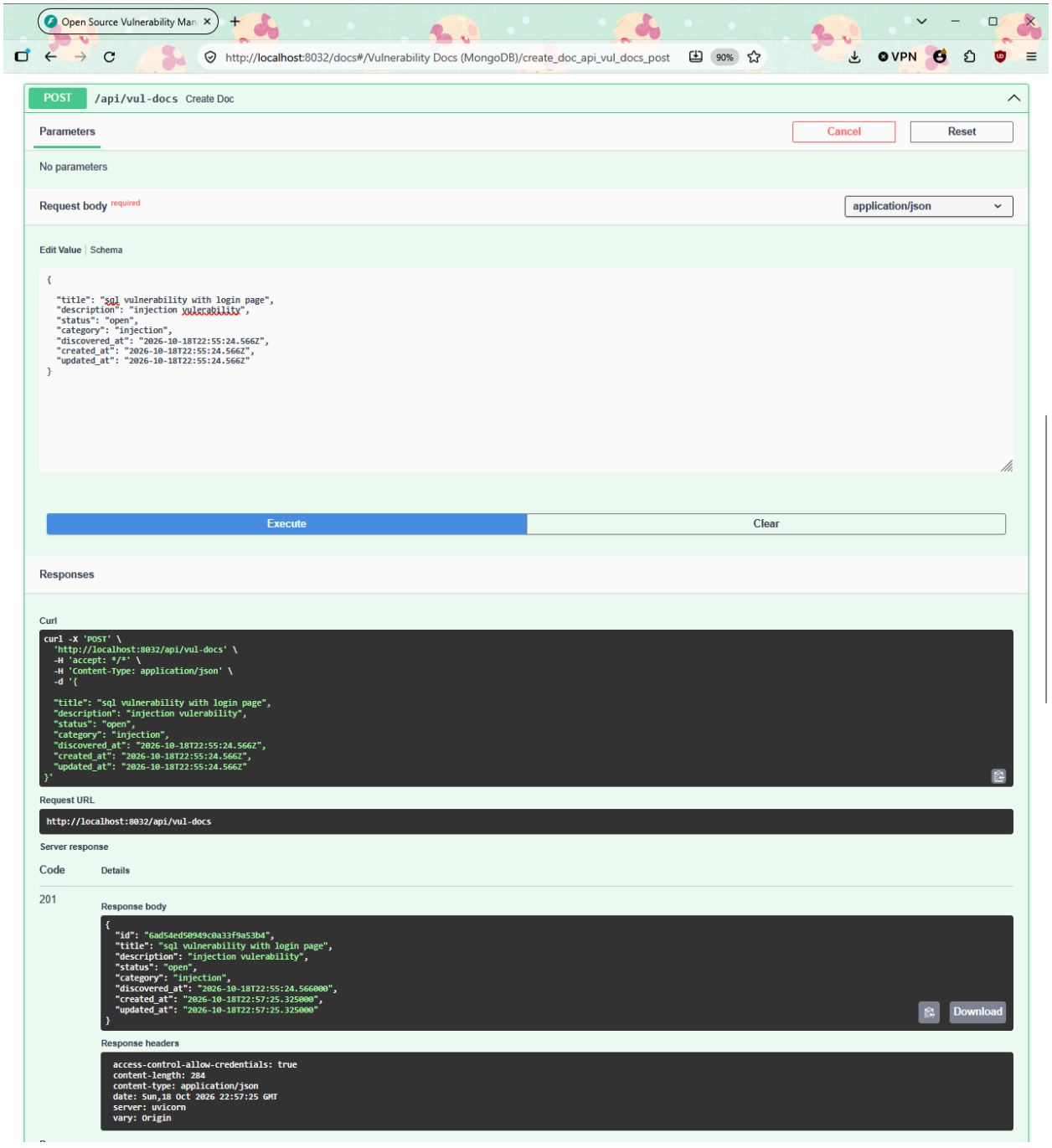
• The schema must include proper validation rules such as required fields, string length limits, and enum constraints.

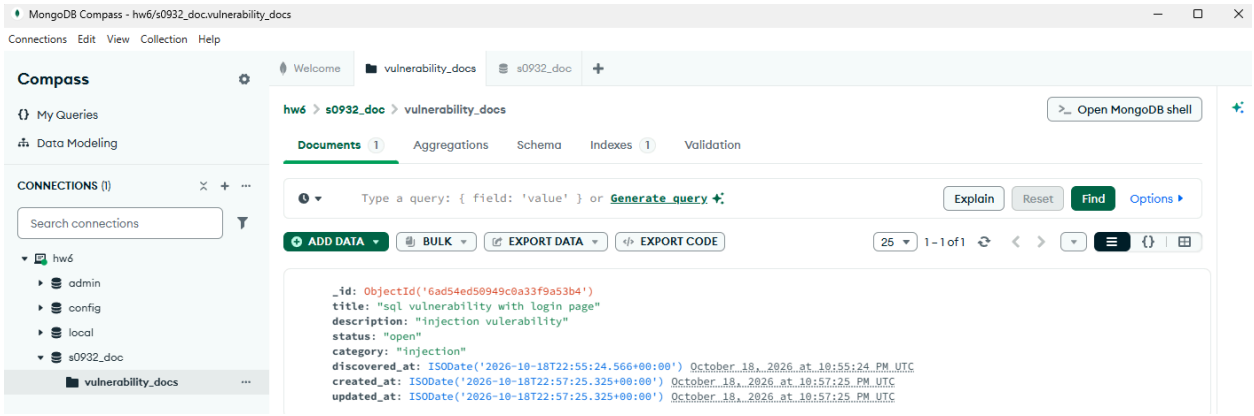
```python
def setup_collection():
    try:
        db = get_db()
        if COLLECTION_NAME in db.list_collection_names():
            return
        db.create_collection(
            COLLECTION_NAME,
            validator={
                "$jsonSchema": {
                    "bsonType": "object",
                    "required": ["title", "status", "category", "discovered_at", "created_at", "updated_at"],
                    "properties": {
                        #string length limit
                        "title": {"bsonType": "string", "minLength": 1, "maxLength": 100},
                        "description": {"bsonType": ["string", "null"]},
                        #enum constraints
                        "status": {"enum": list(get_args(StatusType))},
                        "category": {"enum": list(get_args(CategoryType))},
                        "discovered_at": {"bsonType": "date"},
                        "created_at": {"bsonType": "date"},
                        "updated_at": {"bsonType": "date"},
                    },
                }
            },
        )
        print("Created collection", COLLECTION_NAME)
    except PyMongoError as e:
        print("MongoDB setup warning:", e)
```

II. API Implementation

• Implement a REST API using FastAPI with complete CRUD operations: create (POST), retrieve all (GET), get a single record by ID (GET /:id), update (PUT /:id), delete (DELETE /:id).

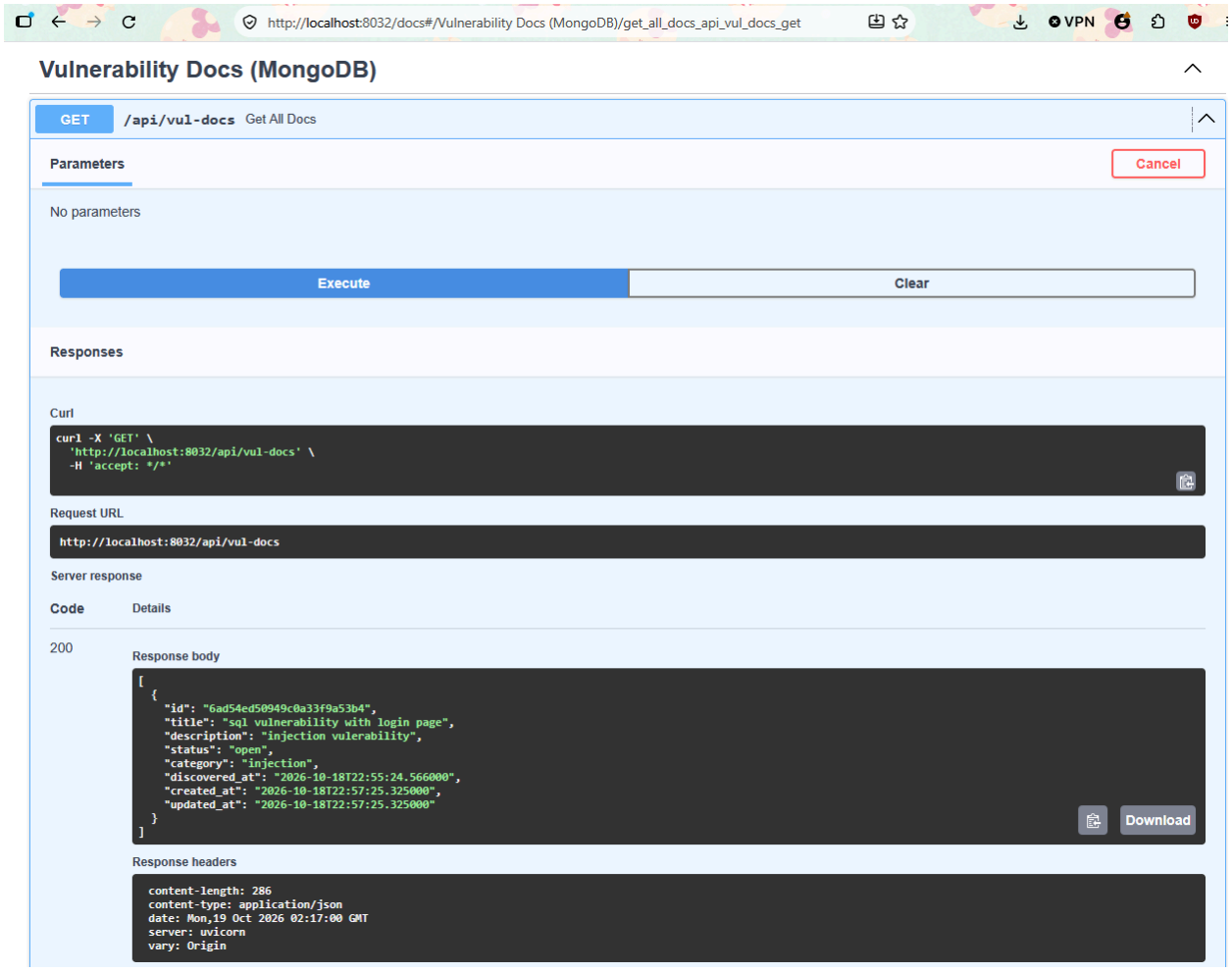
Successful CRUD Operations (Create)


Open Source Vulnerability Man
http://localhost:8032/docs#/Vulnerability Docs (MongoDB)/create_doc_api_vul_docs_post
90%
VPN
POST
/api/vul-docs
Create Doc
Parameters
Cancel
Reset
No parameters
Request body
required
application/json
Edit Value | Schema
{
"title": "sql vulnerability with login page",
"description": "injection vulerability",
"status": "open",
"category": "injection",
"discovered_at": "2026-10-18T22:55:24.566Z",
"created_at": "2026-10-18T22:55:24.566Z",
"updated_at": "2026-10-18T22:55:24.566Z"
}
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://localhost:8032/api/vul-docs' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
"title": "sql vulnerability with login page",
"description": "injection vulerability",
"status": "open",
"category": "injection",
"discovered_at": "2026-10-18T22:55:24.566Z",
"created_at": "2026-10-18T22:55:24.566Z",
"updated_at": "2026-10-18T22:55:24.566Z"
}'
Request URL
http://localhost:8032/api/vul-docs
Server response
Code
Details
201
Response body
{
"id": "6ad54ed50949c0a33f9a53b4",
"title": "sql vulnerability with login page",
"description": "injection vulerability",
"status": "open",
"category": "injection",
"discovered_at": "2026-10-18T22:55:24.566000",
"created_at": "2026-10-18T22:57:25.325000",
"updated_at": "2026-10-18T22:57:25.325000"
}
Download
Response headers
access-control-allow-credentials: true
content-length: 284
content-type: application/json
date: Sun,18 Oct 2026 22:57:25 GMT
server: uvicorn
vary: Origin



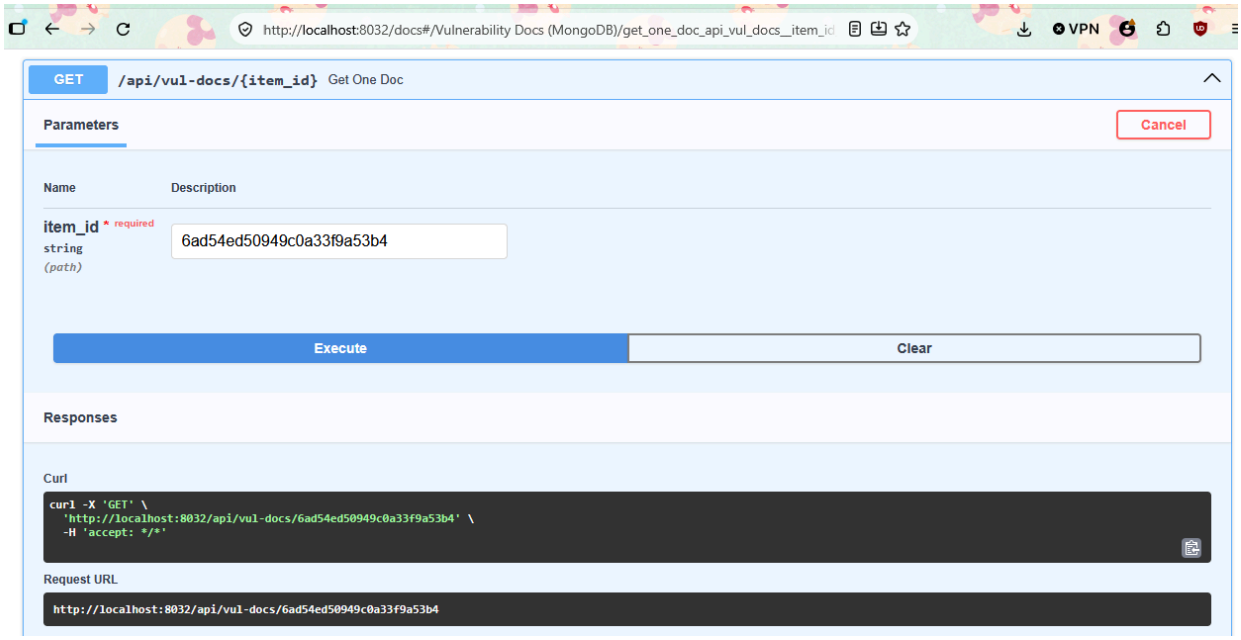

```python
@router.post("", response_model=VulDocOut, status_code=201)
def create_doc(payload: VulDocCreate):
    col = get_collection()
    doc = payload.model_dump()
    doc["created_at"] = now_utc()
    doc["updated_at"] = doc["created_at"]
    try:
        result = col.insert_one(doc)
        return to_out(col.find_one({"_id": result.inserted_id}))
    except PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while creating record")
```

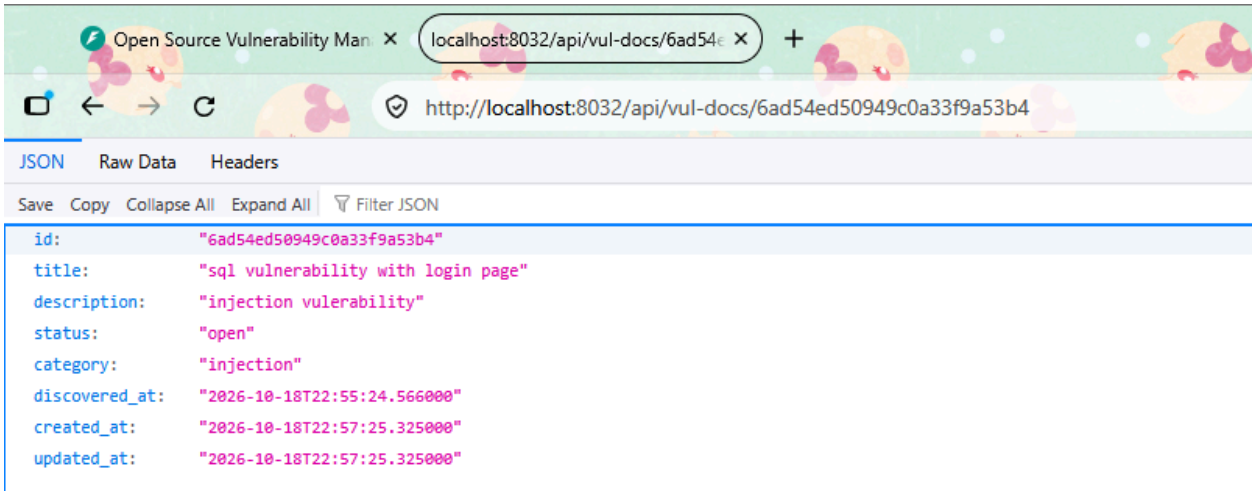
Successful CRUD Operations (Retrieve)




```python
@router.get("", response_model=list[VulDocOut])
def get_all_docs():
    try:
        return [to_out(d) for d in get_collection().find().sort("created_at", -1)]
    except PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while fetching records")
```

Successful CRUD Operations (Get)

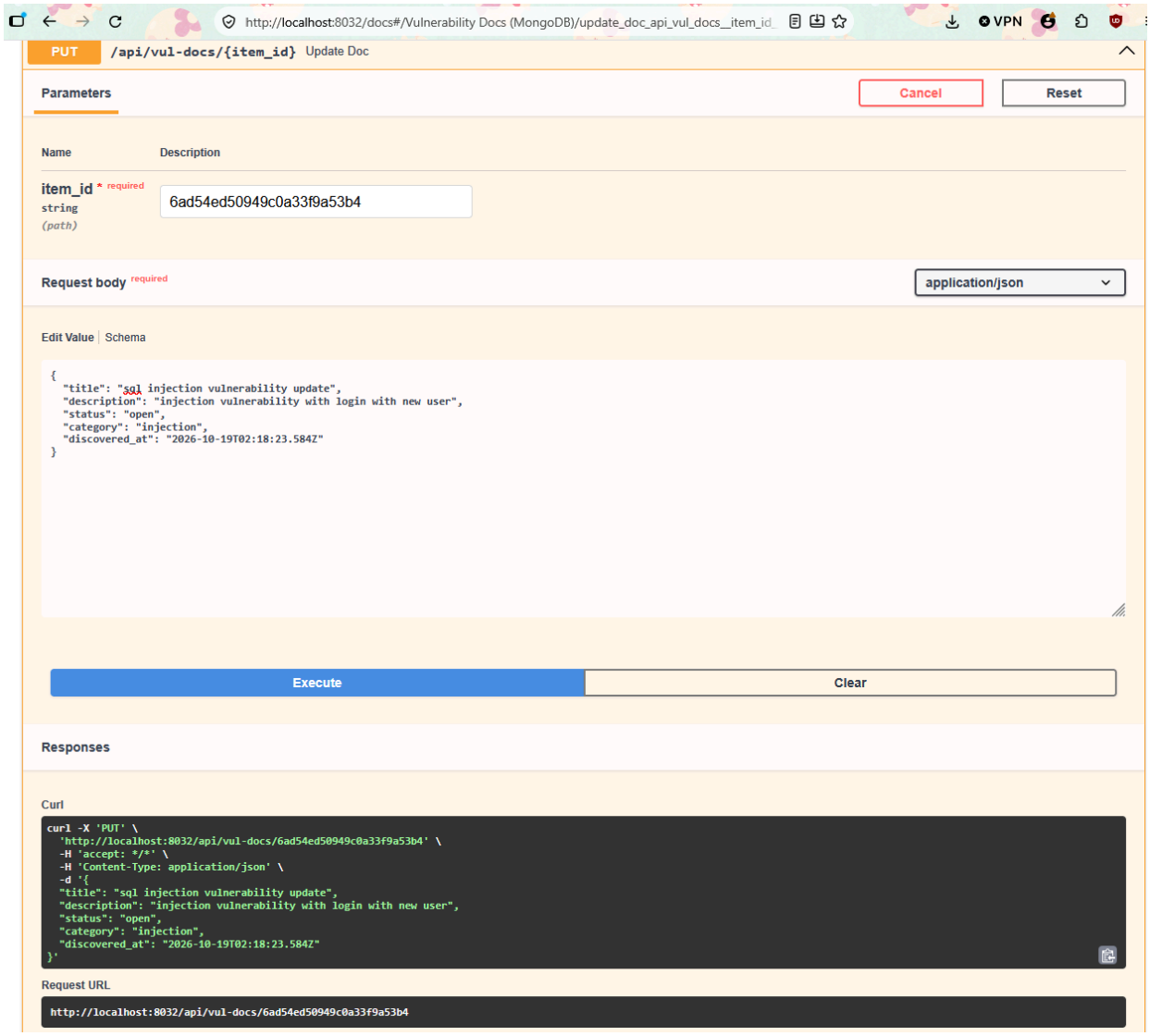





```python
@router.get("/{item_id}", response_model=VulDocOut)
def get_one_doc(item_id: str):
    oid = parse_id(item_id)
    try:
        doc = get_collection().find_one({"_id": oid})
    except PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while fetching record")
    if not doc:
        raise HTTPException(status_code=404, detail="Record not found")
    return to_out(doc)

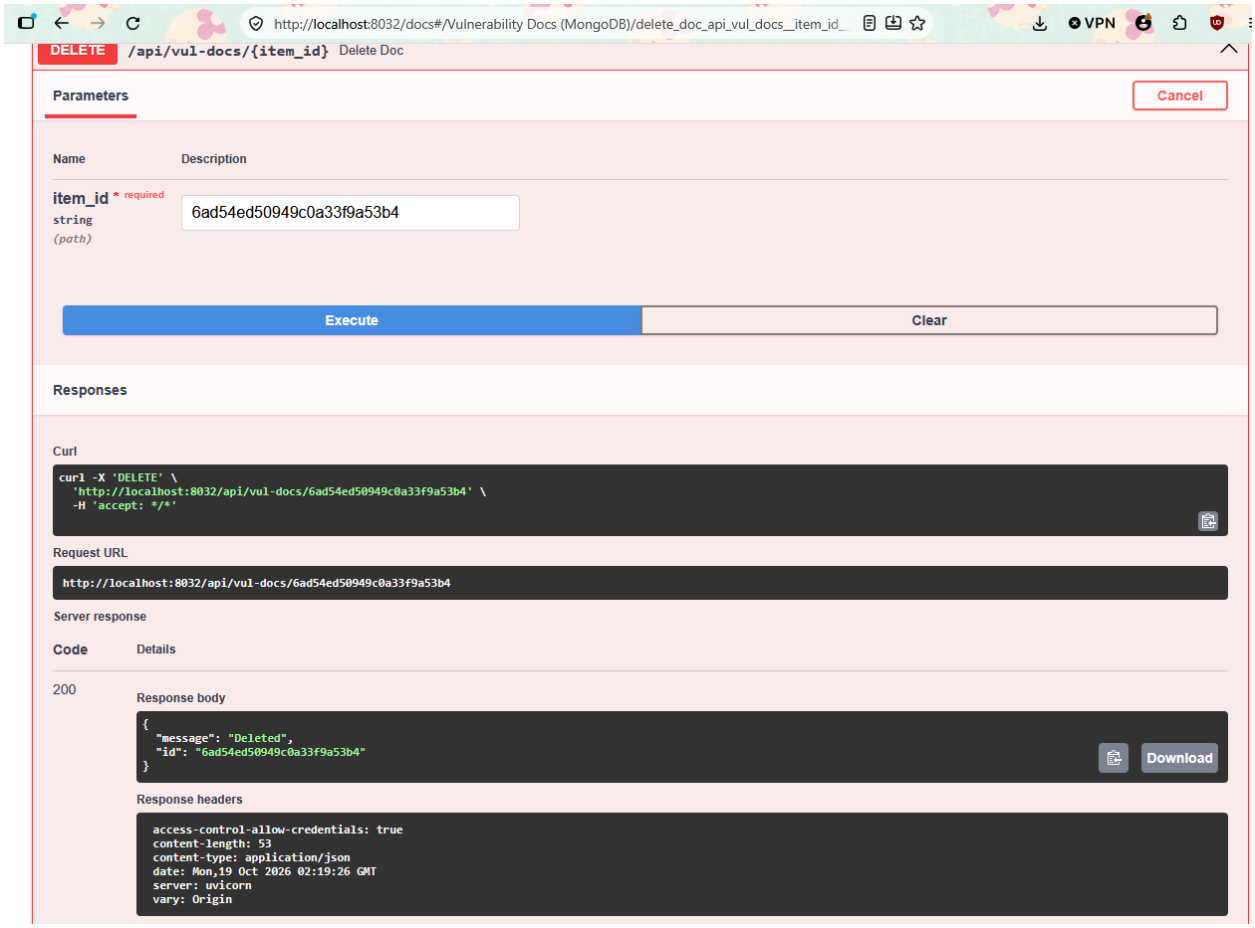
```

Successful CRUD Operations (Update)



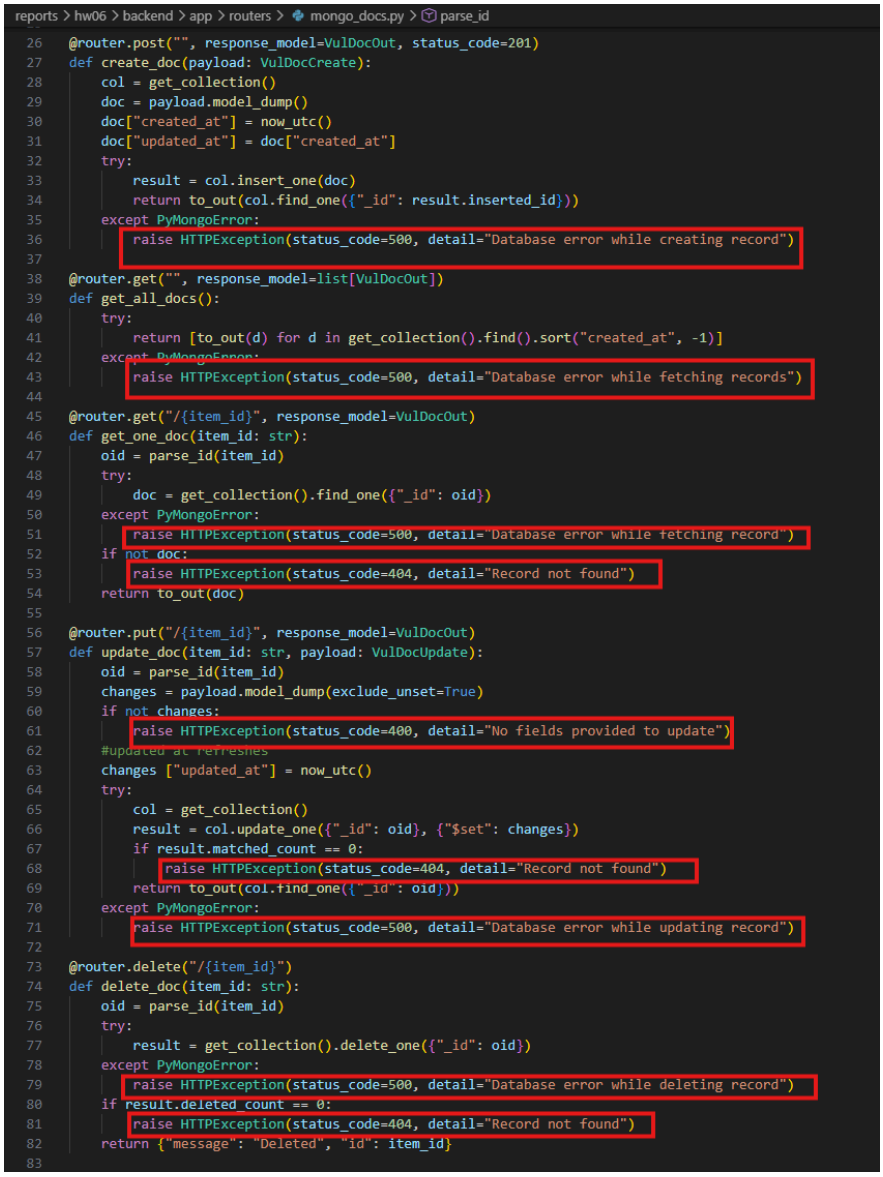

```python
@router.put("/{item_id}", response_model=VulDocOut)
def update_doc(item_id: str, payload: VulDocUpdate):
    oid = parse_id(item_id)
    changes = payload.model_dump(exclude_unset=True)
    if not changes:
        raise HTTPException(status_code=400, detail="No fields provided to update")
    #updated at refreshes
    changes ["updated_at"] = now_utc()
    try:
        col = get_collection()
        result = col.update_one({"_id": oid}, {"$set": changes})
        if result.matched_count == 0:
            raise HTTPException(status_code=404, detail="Record not found")
        return to_out(col.find_one({"_id": oid}))
    except PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while updating record")
```

Successful CRUD Operations (Delete)




```python
@router.delete("/{item_id}")
def delete_doc(item_id: str):
    oid = parse_id(item_id)
    try:
        result = get_collection().delete_one({"_id": oid})
    except PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while deleting record")
    if result.deleted_count == 0:
        raise HTTPException(status_code=404, detail="Record not found")
    return {"message": "Deleted", "id": item_id}
```

• Include proper error handling with appropriate HTTP status codes for scenarios like invalid data, resource not found, and server errors.



• Use middleware for CORS and JSON parsing for request bodies. Structure your code with separate files for server configuration, PyMongo database connection, data models/schemas (Pydantic), and route handlers.

Middleware

reports > hw06 > backend > app > main.py > ...

```python
from .mongo import setup_collection
from .routers.mongo_docs import router as docs_router

Base.metadata.create_all(bind=engine)

app = FastAPI(title="Open Source Vulnerability Management API")

# Allow React dev server
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
app.include_router(auth_router)

setup_collection()
app.include_router(docs_router)
```

Server config

reports > hw06 > backend > app > mongo.py > ...

```python
from typing import get_args
from pymongo import MongoClient
from pymongo.errors import PyMongoError
from .schema import StatusType, CategoryType
import os
from typing import get_args
from dotenv import load_dotenv
from pymongo import MongoClient
from pymongo.errors import PyMongoError
from .mongo_schema import StatusType, CategoryType

load_dotenv()

SID4 = "0932"
PREFIX = "s" + SID4

MONGODB_URI = os.getenv("MONGODB_URI", "mongodb://localhost:27017")
MONGODB_DB = os.getenv("MONGODB_DB", PREFIX + "_doc")
COLLECTION_NAME = "vulnerability_docs"

MESSAGES_COLLECTION = "messages"
SUMMARIES_COLLECTION = "summaries"
EPISODES_COLLECTION = "episodes"
client = MongoClient(MONGODB_URI)
```

Schema/Models

reports > hw06 > backend > app > mongo_schema.py > VulDocOut

```python
from pydantic import BaseModel, Field, EmailStr
from datetime import datetime
from typing import Optional, Literal

StatusType = Literal["open", "triaged", "fixed", "wont_fix"]
CategoryType = Literal["injection", "xss", "auth", "dos", "other"]

class VulDocCreate(BaseModel):
    title: str = Field(min_length=1, max_length=100)
    description: Optional[str] = None
    status: StatusType = "open"
    category: CategoryType
    discovered_at: datetime

class VulDocUpdate(BaseModel):
    title: Optional[str] = Field(default=None, min_length=1, max_length=100)
    description: Optional[str] = None
    status: Optional[StatusType] = None
    category: Optional[CategoryType] = None
    discovered_at: Optional[datetime] = None

class VulDocOut(BaseModel):
    id: str
    title: str
    description: Optional[str] = None
    status: StatusType
    category: CategoryType
    discovered_at: datetime
    created_at: datetime
    updated_at: datetime
```

Routes (the crud routes were included above)

reports > hw06 > backend > app > routers > mongo_docs.py > ...

```python
from datetime import datetime, timezone
from bson import ObjectId
from bson.errors import InvalidId
from fastapi import APIRouter, HTTPException
from pymongo.errors import PyMongoError

from ..mongo import get_collection
from ..mongo_schema import VulDocCreate, VulDocUpdate, VulDocOut

router = APIRouter(prefix="/api/vul-docs", tags=["Vulnerability Docs (MongoDB)"])

def now_utc():
    return datetime.now(timezone.utc)

def to_out(doc: dict) -> dict:
    doc["id"] = str(doc.pop("_id"))
    return doc

def parse_id(item_id: str) -> ObjectId:
    try:
        return ObjectId(item_id)
    except (InvalidId, TypeError):
        raise HTTPException(status_code=400, detail="Invalid id format")

@router.post("", response_model=VulDocOut, status_code=201)
def create_doc(payload: VulDocCreate):
    col = get_collection()
    doc = payload.model_dump()
    doc["created_at"] = now_utc()
    doc["updated_at"] = doc["created_at"]
    try:
        result = col.insert_one(doc)
        return to_out(col.find_one({"_id": result.inserted_id}))
    except PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while creating record")
```

III. Data Validation

• Implement validation using Pydantic schemas to enforce data integrity including required fields, string lengths, enum values, and proper date formats.

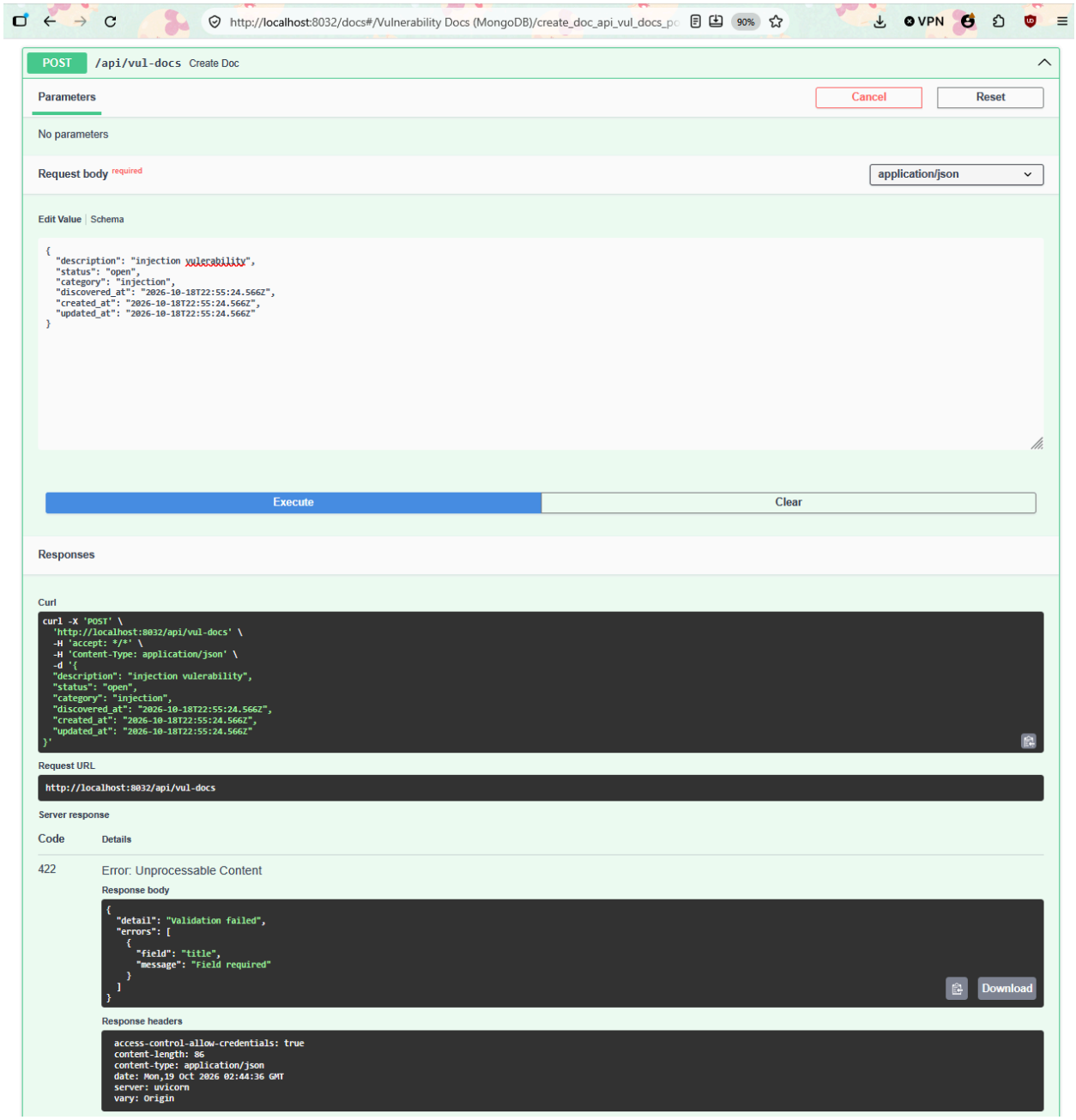
```python
class VulDocCreate(BaseModel):
    title: str = Field(min_length=1, max_length=100)
    description: Optional[str] = None
    status: StatusType = "open"
    category: CategoryType
    discovered_at: datetime
```

• Ensure that validation errors return clear error messages with appropriate HTTP status codes.

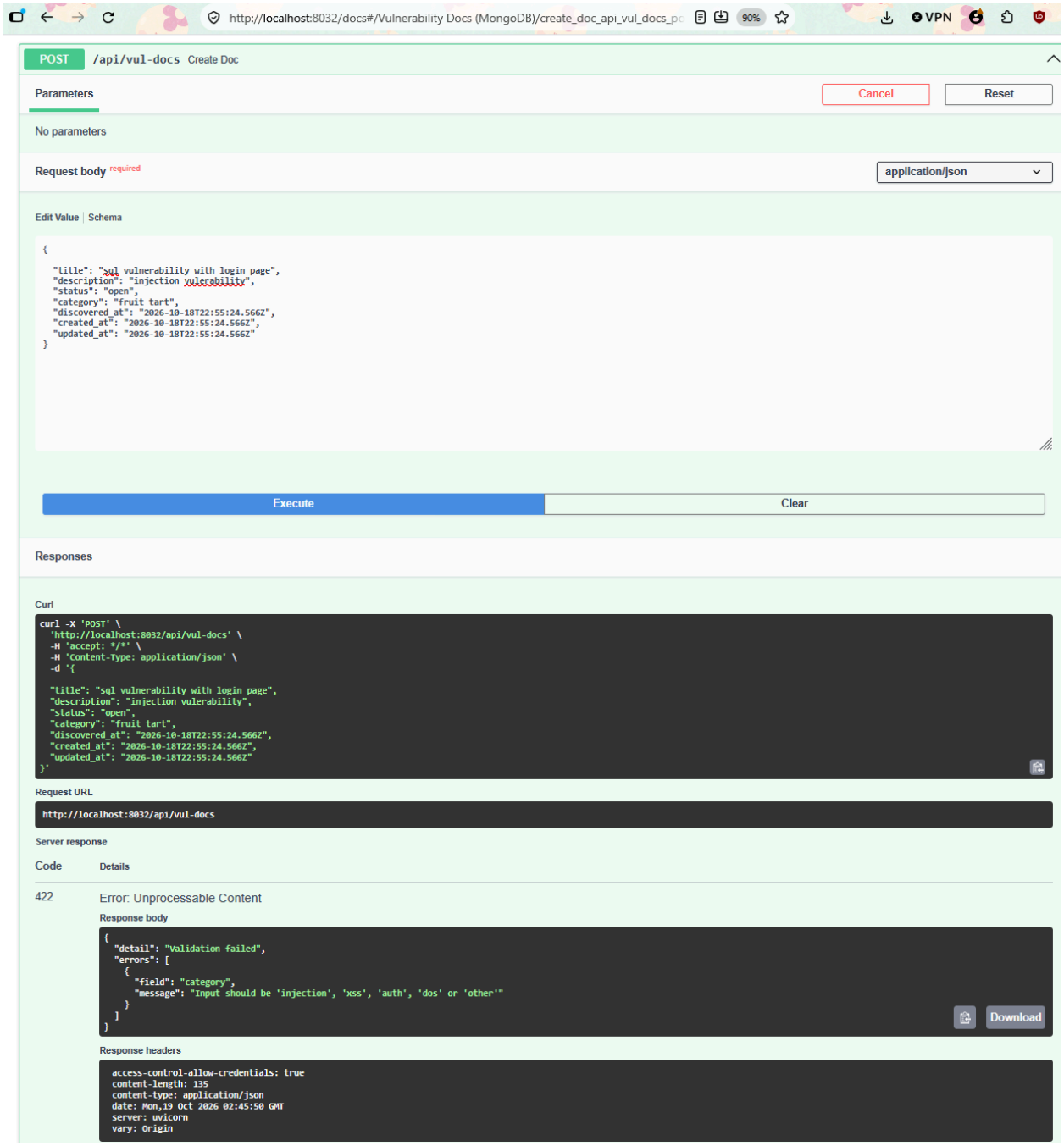
```python
@router.post("", response_model=VulDocOut, status_code=201)
def create_doc(payload: VulDocCreate):
    col = get_collection()
    doc = payload.model_dump()
    doc["created_at"] = now_utc()
    doc["updated_at"] = doc["created_at"]
    try:
        result = col.insert_one(doc)
        return to_out(col.find_one({"_id": result.inserted_id}))
    except PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while creating record")
```

• Test validation by attempting to create records with invalid data such as missing required fields, invalid enum values, and incorrect data types.

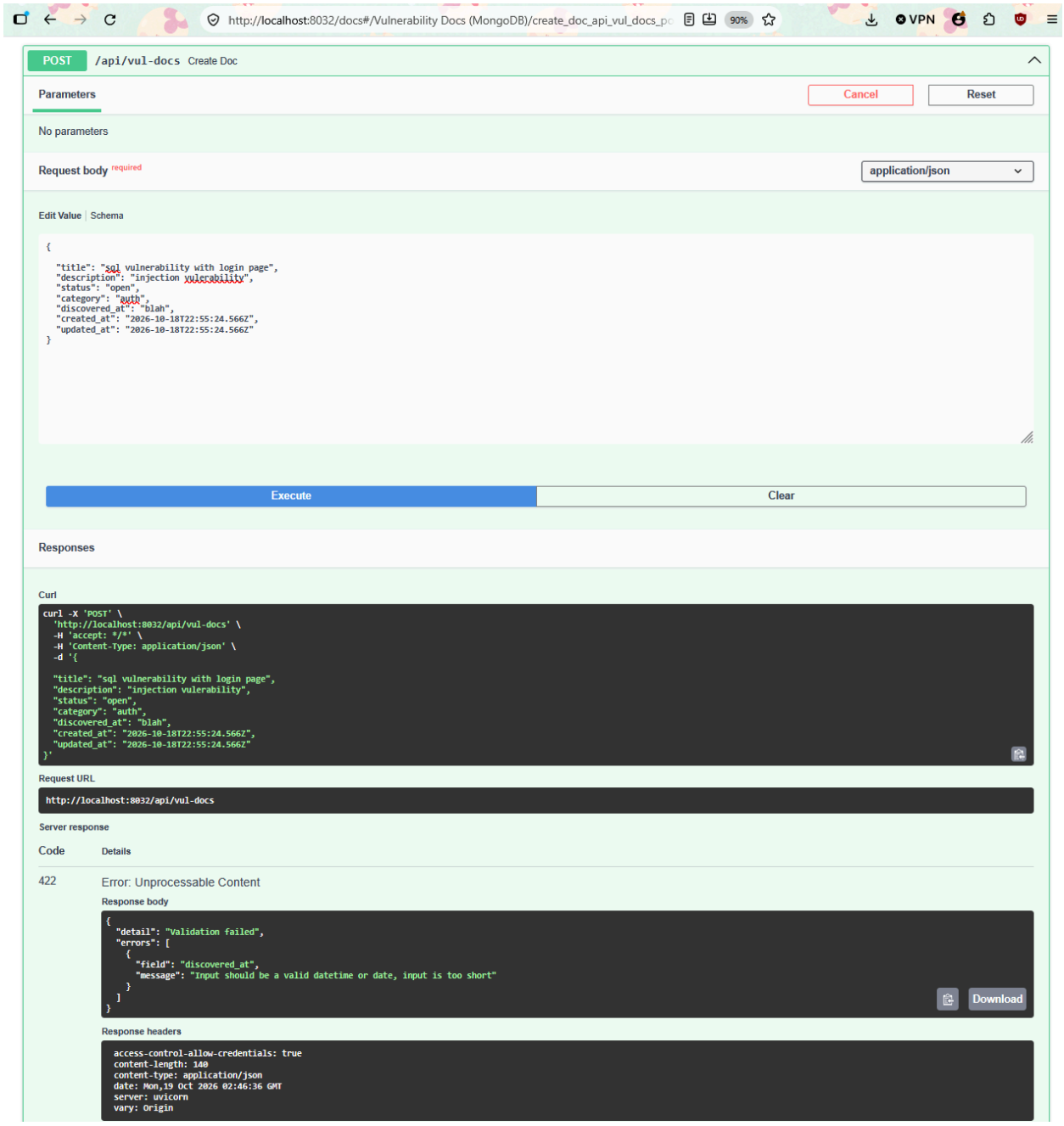
Missing title field



Invalid enum



Incorrect data type



Part 2 - Short-, Long-, and Episodic Memory

Use the same MongoDB database with new collections: messages, summaries, episodes.

Keep your Part 1 collection unchanged - you are only reusing the same database/connection.

MongoDB collections (reuse the same DB)

Create new collections: (3 points)

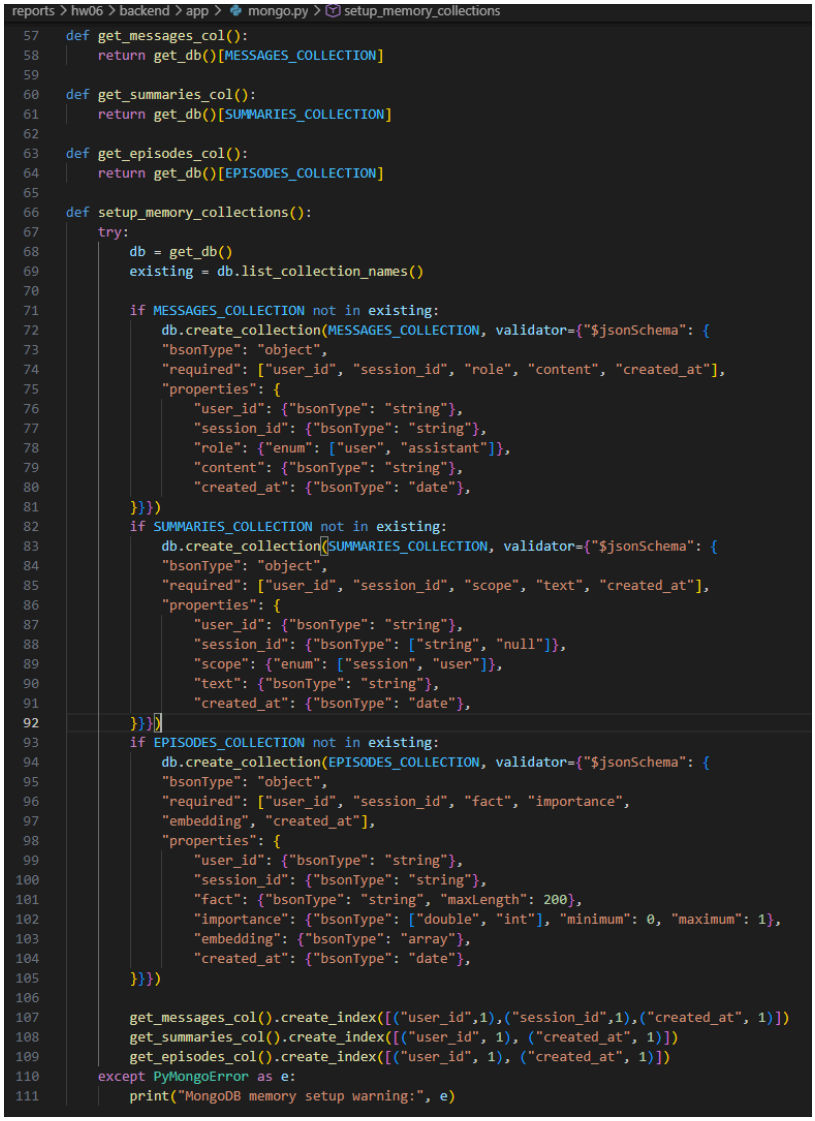
messages

Fields: user_id, session_id, role (user/assistant), content, created_at.

Summaries

Fields: user_id, session_id (nullable for lifetime), scope ('session' | 'user'), text,

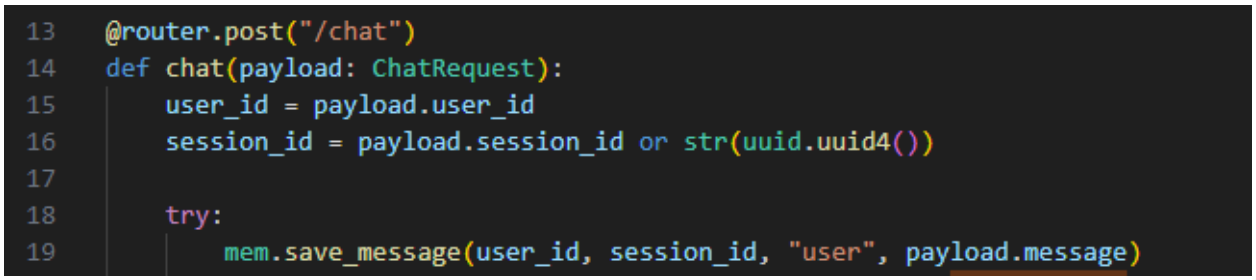
created_at.
episodes
Fields: user_id, session_id, fact (short text), importance (0..1), embedding (array of floats), created_at.



FastAPI Endpoints

1. POST /api/chat

• Request: { user_id, session_id (optional), message }



• Server steps: save the user message to messages; load the short-term window (last N messages, N = SHORT_TERM_N); load the latest session summary and lifetime summary from summaries; retrieve the top-k relevant episodic facts; compose the prompt (system primer + long-term summaries + short-term window + current message + episodic facts); call the model through your HW1 model adapter (local Ollama model); save the assistant reply to messages; every SUMMARIZE_EVERY_USER_MSGS user messages, generate/update a session summary, and occasionally condense session summaries into a lifetime summary.

```python
@router.post("/chat")
def chat(payload: ChatRequest):
    user_id = payload.user_id
    session_id = payload.session_id or str(uuid.uuid4())

    try:
        mem.save_message(user_id, session_id, "user", payload.message)
        window = mem.get_short_term(user_id, session_id, SHORT_TERM_N + 1)[ :- 1]
        session_summary = mem.get_latest_summary(user_id, "session", session_id)
        lifetime_summary = mem.get_latest_summary(user_id, "user")
        facts = mem.retrieve_episodes(user_id, payload.message)
        prompt = mem.build_prompt(session_summary, lifetime_summary,
        window, payload.message, facts)
        reply, prompt_tokens, completion_tokens = llm_adapter.chat(prompt)
        mem.save_message(user_id, session_id, "assistant", reply)
        summary_created = mem.maybe_summarize(user_id, session_id)
        mem.save_episodes(user_id, session_id, mem.extract_facts(payload.message))

    except RuntimeError as e:
        raise HTTPException(status_code=502, detail=str(e))
    except PyMongoError:
        raise HTTPException(status_code=500, detail="Database error in chat")
```

• Response: the assistant reply plus an object showing what was used - short-term message count/list, the long-term summary text (if any), and the episodic facts retrieved for this turn.

```python
return {
    "reply": reply,
    "session_id": session_id,
    "memory_used": {
        "short_term_count": len(window),
        "short_term_messages": window,
        "session_summary": session_summary,
        "lifetime_summary": lifetime_summary,
        "episodic_facts": facts,
        "new_summary_created": summary_created,
    },
    "usage": {"prompt_tokens": prompt_tokens, "completion_tokens": completion_tokens},
}
```

2. GET /api/memory/{user_id} - returns the last ~16 messages, latest session summary,

latest lifetime summary, and last ~20 episodic facts.

```python
@router.get("/memory/{user_id}")
def get_memory(user_id: str):
    try:
        msgs = list(get_messages_col().find({"user_id": user_id},{"_id": 0})
        .sort([("created_at", -1), ("_id", -1)]).limit(16))
        msgs.reverse()
        #latest session summ with latest lifetime summ
        session_doc = get_summaries_col().find_one(
        {"user_id": user_id, "scope": "session"}, {"_id": 0},
        sort=[("created_at", -1),("_id", -1)])
        lifetime_doc = get_summaries_col().find_one(
        {"user_id": user_id, "scope": "user"}, {"_id": 0},
        sort=[("created_at", -1),("_id", -1)])
        #last episodic facts
        episodes = list(get_episodes_col().find({"user_id": user_id}, {"_id": 0, "embedding": 0})
        .sort([("created_at",-1),("_id",-1)]).limit(20))
    except PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while reading memory")
    return {"user_id": user_id, "messages": msgs,
        "latest_session_summary": session_doc,
        "latest_lifetime_summary": lifetime_doc,
        "episodic_facts": episodes}
```

3. GET /api/aggregate/{user_id} - returns daily message counts (grouped by date) and a small list of the most recent summaries.

```python
@router.get("/aggregate/{user_id}")
def get_aggregate(user_id: str):
    try:
        pipeline = [
            {"$match": {"user_id": user_id}},
            {"$group": {"_id": {"$dateToString": {"format": "%Y-%m-%d", "date": "$created_at"}},
            "count": {"$sum": 1}}},
            {"$sort": {"_id": 1}},
        ]
        daily = [{"date": r["_id"], "count": r["count"]} for r in get_messages_col().aggregate(pipeline)]
        #small list of the most recent summaries
        recent = list(get_summaries_col().find({"user_id": user_id}, {"_id": 0})
    .sort([("created_at",-1),("_id",-1)]).limit(5))
    except PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while aggregating")
    return {"user_id": user_id, "daily_message_counts": daily, "recent_summaries": recent}
```

Memory logic

• Short-term memory: sliding window of the last N messages for the session.

```python
def save_message(user_id, session_id, role, content):
    get_messages_col().insert_one({
        "user_id": user_id, "session_id": session_id,
        "role": role, "content": content, "created_at": now_utc(),
    })

def get_short_term(user_id, session_id, n):
    rows = list(get_messages_col().find({"user_id": user_id, "session_id": session_id}, {"_id": 0, "role": 1, "content": 1}).sort([("created_at", -1), ("_id", -1)]).limit(n))
    rows.reverse()
    return rows
```

• Long-term summaries: trigger every M user messages; summarize recent conversation into concise bullets under scope='session'; build/refresh a lifetime summary by condensing multiple session summaries under scope='user' with session_id = null.

```python
def get_latest_summary(user_id, scope, session_id=None):
    query = {"user_id": user_id, "scope": scope}
    if scope == "session":
        query["session_id"] = session_id
    doc = get_summaries_col().find_one(query, {"_id": 0}, sort=[("created_at", -1), ("_id", -1)])
    return doc["text"] if doc else None

def save_summary(user_id, session_id, scope, text):
    get_summaries_col().insert_one({
        "user_id": user_id,
        "session_id": session_id,
        "scope": scope, "text": text, "created_at": now_utc(),
    })

def maybe_summarize(user_id, session_id):
    user_msg_count = get_messages_col().count_documents(
    {"user_id": user_id, "session_id": session_id, "role": "user"})
    if user_msg_count == 0 or user_msg_count % SUMMARIZE_EVERY_USER_MSGS != 0:
        return False

    recent = get_short_term(user_id, session_id, SUMMARIZE_EVERY_USER_MSGS * 2)
    transcript = "\n".join(m["role"] + ": " + m["content"] for m in recent)
    prompt = ("Summarize this conversation into 3-5 concise bullet points."
        "Keep names, numbers and decisions. \n\n" + transcript)
    text, _, _= llm_adapter.chat([{"role": "user", "content": prompt}])
    save_summary(user_id, session_id, "session", text)
    session_summary_count = get_summaries_col().count_documents(
        {"user_id": user_id, "scope": "session"})
    if session_summary_count % CONDENSE_EVERY_N_SUMMARIES == 0:
        condense_lifetime(user_id)
    return True
```

• Episodic memory: extract up to 3 short facts per user message with an importance score (0..1); embed each fact and store the array in MongoDB; for each turn, retrieve the top-k relevant episodes by cosine similarity between the current message embedding and stored episodes.

```python
def extract_facts(user_message):
    prompt = ("Extract up to 3 short facts worth remembering from the user message below."
        'Reply with ONLY a JSON list like [{"fact": " ... ", "importance": 0.7}].'
        "importance is a number from 0 to 1."
        "If nothing is worth remembering reply with [].\n\n"
        "User message: " + user_message)
    text, _, _= llm_adapter.chat([{"role": "user", "content": prompt}])
    start = text.find("[")
    end = text.rfind("]")
    if start == -1 or end == -1:
        return []
    try:
        items = json.loads(text[start:end + 1])
    except json.JSONDecodeError:
        return []

    facts = []
    for item in items[:3]:
        if not isinstance(item, dict) or not item.get("fact"):
            continue
        try:
            importance = float(item.get("importance", 0.5))
        except (TypeError, ValueError):
            importance = 0.5
        importance = max(0.0, min(1.0, importance))
        facts.append({"fact": str(item["fact"])[:200], "importance": importance})
    return facts
```

```python
def save_episodes(user_id, session_id, facts):
    docs = []
    for f in facts:
        docs.append ({
            "user_id": user_id, "session_id": session_id,
            "fact": f["fact"], "importance": f["importance"],
            "embedding": llm_adapter.embed(f["fact"]),
            "created_at": now_utc(),
        })
    if docs:
        get_episodes_col().insert_many(docs)

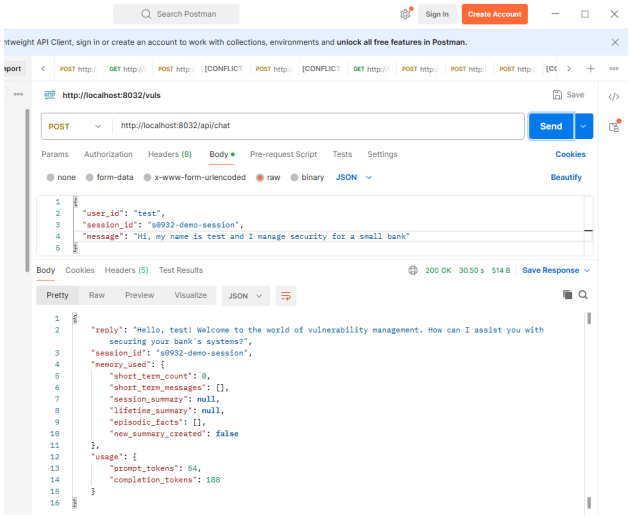
def cosine_similarity(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    len_a = math.sqrt(sum(x * x for x in a))
    len_b = math.sqrt(sum(y * y for y in b))
    if len_a == 0 or len_b == 0:
        return 0.0
    return dot / (len_a * len_b)

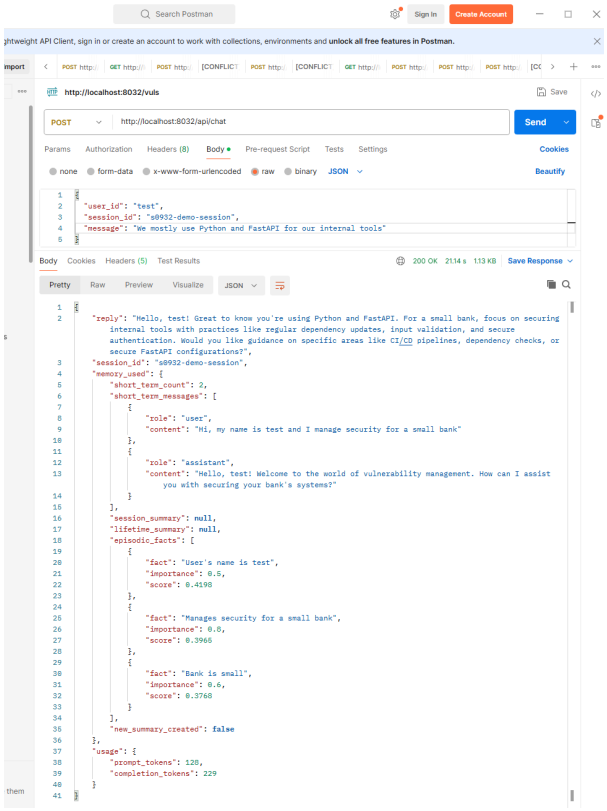
def retrieve_episodes(user_id, message, k=TOP_K):
    episodes = list(get_episodes_col().find({"user_id": user_id}))
    if not episodes:
        return []
    query_vec = llm_adapter.embed (message)
    scored = []
    for ep in episodes:
        score = cosine_similarity(query_vec, ep["embedding"])
        scored.append({"fact": ep["fact"], "importance": ep["importance"],
            "score": round(score, 4)})
    scored.sort(key=lambda e: e["score"], reverse=True)# best match first
    return scored[:k]
```

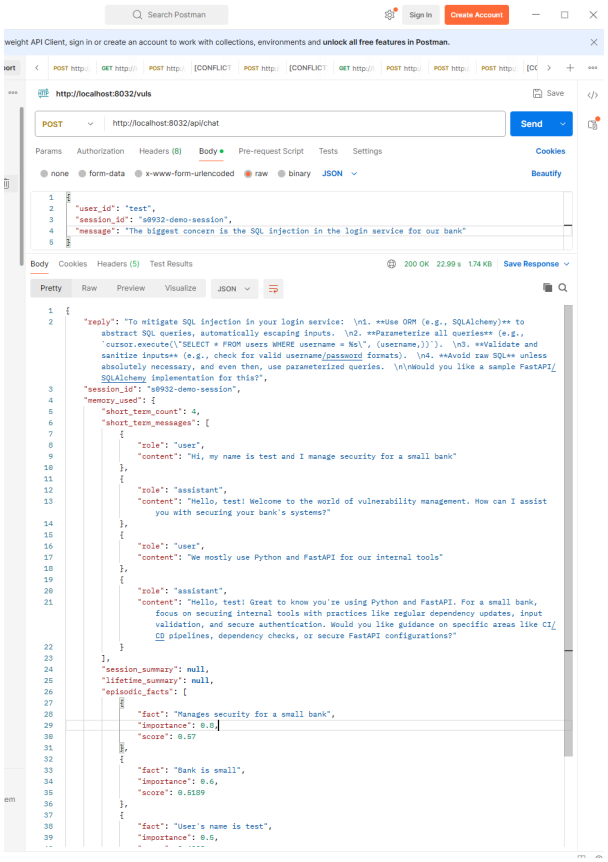
How to run and test

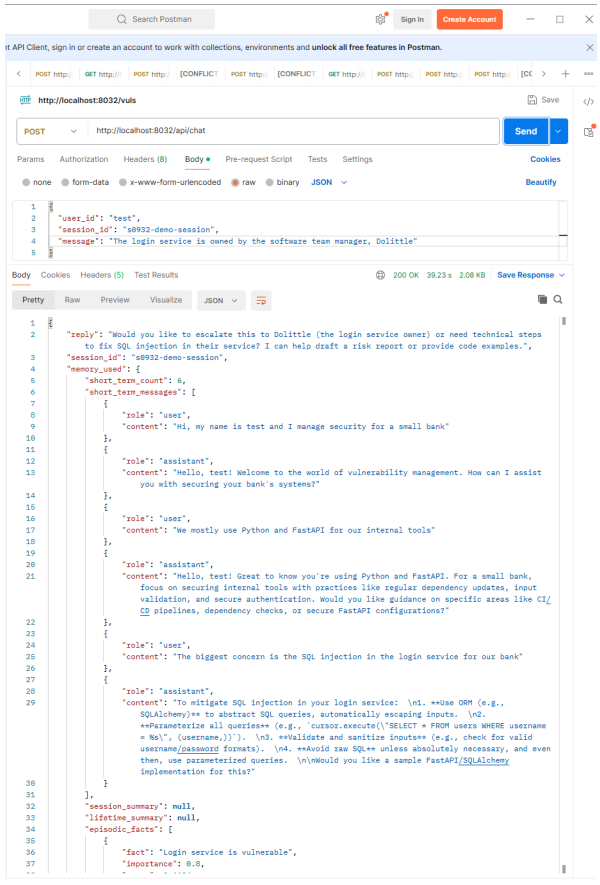
Send at least 10 messages in one session (via Postman/cURL) so the short-term window, summary generation, and episodic retrieval can all be observed. Optionally build a simple HTML page with one input box calling POST /api/chat to simulate a mini chat interface - not required, but recommended for screenshots.

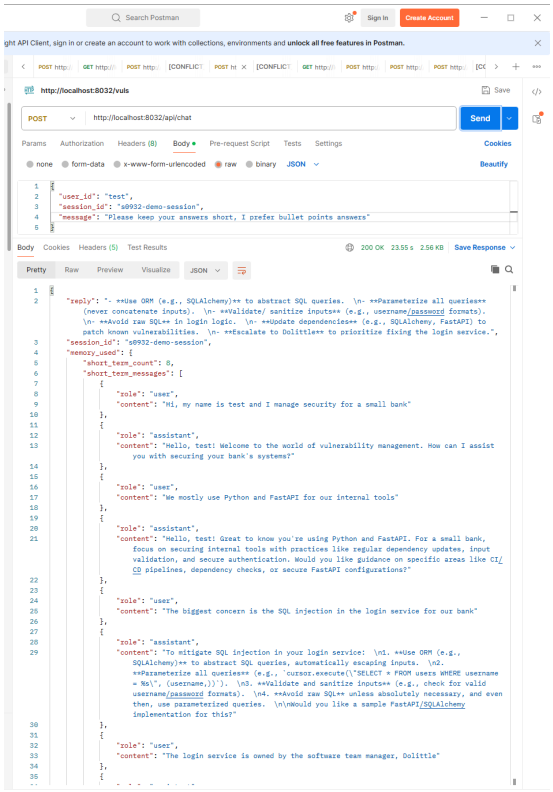
a conversation screenshot showing the assistant's replies


Search Postman
Sign In
Create Account
http://localhost:8032/vuls
POST
http://localhost:8032/api/chat
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
{
"user_id": "test",
"session_id": "s0932-demo-session",
"message": "Hi, my name is test and I manage security for a small bank"
}
Body
Cookies
Headers (5)
Test Results
200 OK
30.50 s
514 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"reply": "Hello, test! Welcome to the world of vulnerability management. How can I assist you with securing your bank's systems?",
"session_id": "s0932-demo-session",
"memory_used": {
"short_term_count": 0,
"short_term_messages": [],
"session_summary": null,
"lifetime_summary": null,
"episodic_facts": [],
"new_summary_created": false
},
"usage": {
"prompt_tokens": 54,
"completion_tokens": 188
}
}


Search Postman
Sign In
Create Account
http://localhost:8032/vuls
POST
http://localhost:8032/api/chat
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
{
"user_id": "test",
"session_id": "s0932-demo-session",
"message": "We mostly use Python and FastAPI for our internal tools"
}
Body
Cookies
Headers (5)
Test Results
200 OK
21.14 s
1.13 KB
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"reply": "Hello, test! Great to know you're using Python and FastAPI. For a small bank, focus on securing internal tools with practices like regular dependency updates, input validation, and secure authentication. Would you like guidance on specific areas like CI/CD pipelines, dependency checks, or secure FastAPI configurations?",
"session_id": "s0932-demo-session",
"memory_used": {
"short_term_count": 2,
"short_term_messages": [
{
"role": "user",
"content": "Hi, my name is test and I manage security for a small bank"
},
{
"role": "assistant",
"content": "Hello, test! Welcome to the world of vulnerability management. How can I assist you with securing your bank's systems?"
}
],
"session_summary": null,
"lifetime_summary": null,
"episodic_facts": [
{
"fact": "User's name is test",
"importance": 0.5,
"score": 0.4198
},
{
"fact": "Manages security for a small bank",
"importance": 0.6,
"score": 0.3965
},
{
"fact": "Bank is small",
"importance": 0.6,
"score": 0.3768
}
],
"new_summary_created": false
},
"usage": {
"prompt_tokens": 128,
"completion_tokens": 229
}
}


Search Postman
Sign In
Create Account
http://localhost:8032/vuls
POST
http://localhost:8032/api/chat
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
{
"user_id": "test",
"session_id": "s0932-demo-session",
"message": "The biggest concern is the SQL injection in the login service for our bank"
}
Body
Cookies
Headers (5)
Test Results
200 OK
22.99 s
1.74 KB
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"reply": "To mitigate SQL injection in your login service: \n1. **Use ORM (e.g., SQLAlchemy)** to abstract SQL queries, automatically escaping inputs. \n2. **Parameterize all queries** (e.g., `cursor.execute(\"SELECT * FROM users WHERE username = %s\", (username,))`). \n3. **Validate and sanitize inputs** (e.g., check for valid username/password formats). \n4. **Avoid raw SQL** unless absolutely necessary, and even then, use parameterized queries. \n\nWould you like a sample FastAPI/SQLAlchemy implementation for this?",
"session_id": "s0932-demo-session",
"memory_used": {
"short_term_count": 4,
"short_term_messages": [
{
"role": "user",
"content": "Hi, my name is test and I manage security for a small bank"
},
{
"role": "assistant",
"content": "Hello, test! Welcome to the world of vulnerability management. How can I assist you with securing your bank's systems?"
},
{
"role": "user",
"content": "We mostly use Python and FastAPI for our internal tools"
},
{
"role": "assistant",
"content": "Hello, test! Great to know you're using Python and FastAPI. For a small bank, focus on securing internal tools with practices like regular dependency updates, input validation, and secure authentication. Would you like guidance on specific areas like CI/CD pipelines, dependency checks, or secure FastAPI configurations?"
}
],
"session_summary": null,
"lifetime_summary": null,
"episodic_facts": [
{
"fact": "Manages security for a small bank",
"importance": 0.6,
"score": 0.57
},
{
"fact": "Bank is small",
"importance": 0.6,
"score": 0.5189
},
{
"fact": "User's name is test",
"importance": 0.5,


Search Postman
Sign In
Create Account
http://localhost:8032/vuls
POST
http://localhost:8032/api/chat
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
{
"user_id": "test",
"session_id": "s0932-demo-session",
"message": "The login service is owned by the software team manager, Dolittle"
}
Body
Cookies
Headers (5)
Test Results
200 OK
39.23 s
2.08 KB
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"reply": "Would you like to escalate this to Dolittle (the login service owner) or need technical steps to fix SQL injection in their service? I can help draft a risk report or provide code examples.",
"session_id": "s0932-demo-session",
"memory_used": {
"short_term_count": 6,
"short_term_messages": [
{
"role": "user",
"content": "Hi, my name is test and I manage security for a small bank"
},
{
"role": "assistant",
"content": "Hello, test! Welcome to the world of vulnerability management. How can I assist you with securing your bank's systems?"
},
{
"role": "user",
"content": "We mostly use Python and FastAPI for our internal tools"
},
{
"role": "assistant",
"content": "Hello, test! Great to know you're using Python and FastAPI. For a small bank, focus on securing internal tools with practices like regular dependency updates, input validation, and secure authentication. Would you like guidance on specific areas like CI/CD pipelines, dependency checks, or secure FastAPI configurations?"
},
{
"role": "user",
"content": "The biggest concern is the SQL injection in the login service for our bank"
},
{
"role": "assistant",
"content": "To mitigate SQL injection in your login service: \n1. **Use ORM (e.g., SQLAlchemy)** to abstract SQL queries, automatically escaping inputs. \n2. **Parameterize all queries** (e.g., `cursor.execute(\"SELECT * FROM users WHERE username = %s\", (username,))`). \n3. **Validate and sanitize inputs** (e.g., check for valid username/password formats). \n4. **Avoid raw SQL** unless absolutely necessary, and even then, use parameterized queries. \n\nWould you like a sample FastAPI/SQLAlchemy implementation for this?"
}
],
"session_summary": null,
"lifetime_summary": null,
"episodic_facts": [
{
"fact": "Login service is vulnerable",
"importance": 0.8,



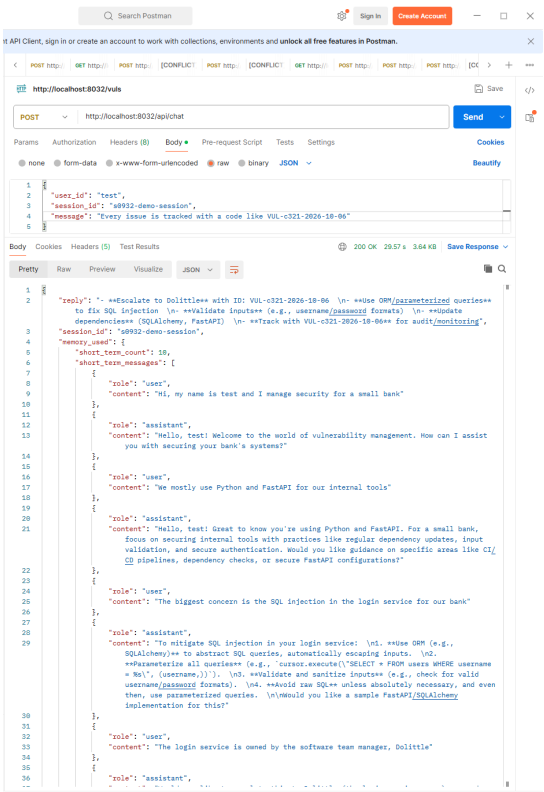
POST http://localhost:8032/api/chat

Body (raw JSON)

```
{
  "user_id": "test",
  "session_id": "s0932-demo-session",
  "message": "Please keep your answers short, I prefer bullet points answers"
}
```

Response (200 OK, 23.55 s, 2.56 KB)

```
{
  "reply": "- **Use ORM (e.g., SQLAlchemy)** to abstract SQL queries.  \n- **Parameterize all queries** (never concatenate inputs).  \n- **Validate/ sanitize inputs** (e.g., username/password formats).  \n- **Avoid raw SQL** in login logic.  \n- **Update dependencies** (e.g., SQLAlchemy, FastAPI) to patch known vulnerabilities.  \n- **Escalate to Dolittle** to prioritize fixing the login service.",
  "session_id": "s0932-demo-session",
  "memory_used": {
    "short_term_count": 8,
    "short_term_messages": [
      {
        "role": "user",
        "content": "Hi, my name is test and I manage security for a small bank"
      },
      {
        "role": "assistant",
        "content": "Hello, test! Welcome to the world of vulnerability management. How can I assist you with securing your bank's systems?"
      },
      {
        "role": "user",
        "content": "We mostly use Python and FastAPI for our internal tools"
      },
      {
        "role": "assistant",
        "content": "Hello, test! Great to know you're using Python and FastAPI. For a small bank, focus on securing internal tools with practices like regular dependency updates, input validation, and secure authentication. Would you like guidance on specific areas like CI/CD pipelines, dependency checks, or secure FastAPI configurations?"
      },
      {
        "role": "user",
        "content": "The biggest concern is the SQL injection in the login service for our bank"
      },
      {
        "role": "assistant",
        "content": "To mitigate SQL injection in your login service:  \n1. **Use ORM (e.g., SQLAlchemy)** to abstract SQL queries, automatically escaping inputs.  \n2. **Parameterize all queries** (e.g., `cursor.execute(\"SELECT * FROM users WHERE username = %s\", (username,))`).  \n3. **Validate and sanitize inputs** (e.g., check for valid username/password formats).  \n4. **Avoid raw SQL** unless absolutely necessary, and even then, use parameterized queries.  \n\nWould you like a sample FastAPI/SQLAlchemy implementation for this?"
      },
      {
        "role": "user",
        "content": "The login service is owned by the software team manager, Dolittle"
      },
      {
```



POST http://localhost:8032/api/chat

Body (raw JSON)

```
{
  "user_id": "test",
  "session_id": "s0932-demo-session",
  "message": "Every issue is tracked with a code like VUL-c321-2026-10-06"
}
```

Response (200 OK, 29.57 s, 3.64 KB)

```
{
  "reply": "- **Escalate to Dolittle** with ID: VUL-c321-2026-10-06  \n- **Use ORM/parameterized queries** to fix SQL injection  \n- **Validate inputs** (e.g., username/password formats)  \n- **Update dependencies** (SQLAlchemy, FastAPI)  \n- **Track with VUL-c321-2026-10-06** for audit/monitoring",
  "session_id": "s0932-demo-session",
  "memory_used": {
    "short_term_count": 10,
    "short_term_messages": [
      {
        "role": "user",
        "content": "Hi, my name is test and I manage security for a small bank"
      },
      {
        "role": "assistant",
        "content": "Hello, test! Welcome to the world of vulnerability management. How can I assist you with securing your bank's systems?"
      },
      {
        "role": "user",
        "content": "We mostly use Python and FastAPI for our internal tools"
      },
      {
        "role": "assistant",
        "content": "Hello, test! Great to know you're using Python and FastAPI. For a small bank, focus on securing internal tools with practices like regular dependency updates, input validation, and secure authentication. Would you like guidance on specific areas like CI/CD pipelines, dependency checks, or secure FastAPI configurations?"
      },
      {
        "role": "user",
        "content": "The biggest concern is the SQL injection in the login service for our bank"
      },
      {
        "role": "assistant",
        "content": "To mitigate SQL injection in your login service:  \n1. **Use ORM (e.g., SQLAlchemy)** to abstract SQL queries, automatically escaping inputs.  \n2. **Parameterize all queries** (e.g., `cursor.execute(\"SELECT * FROM users WHERE username = %s\", (username,))`).  \n3. **Validate and sanitize inputs** (e.g., check for valid username/password formats).  \n4. **Avoid raw SQL** unless absolutely necessary, and even then, use parameterized queries.  \n\nWould you like a sample FastAPI/SQLAlchemy implementation for this?"
      },
      {
        "role": "user",
        "content": "The login service is owned by the software team manager, Dolittle"
      },
      {
        "role": "assistant",
```



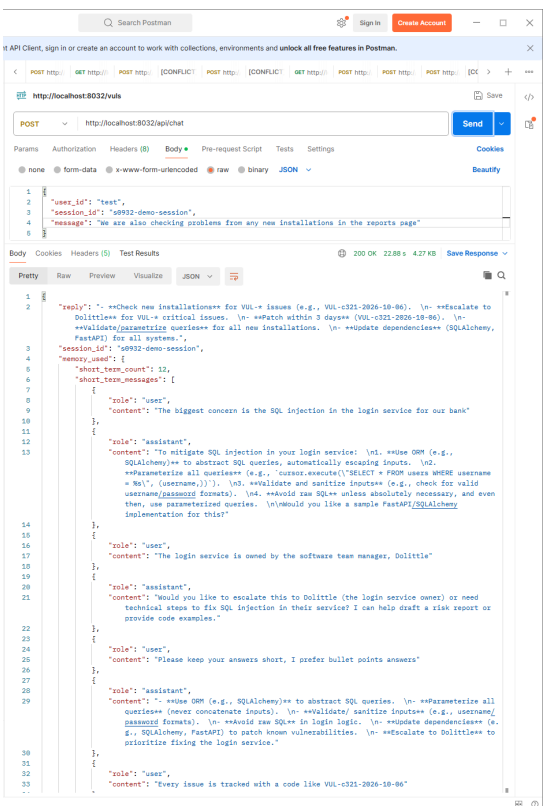
POST http://localhost:8032/api/chat

Body (raw JSON)

```
{
  "user_id": "test",
  "session_id": "s0932-demo-session",
  "message": "Our patch deadline for critical vulnerabilities is 3 days"
}
```

Response (200 OK, 16.73 s, 4.27 KB)

```
{
  "reply": "- **Escalate to Dolittle** (VUL-c321-2026-10-06) **immediately**  \n- **Fix SQL injection** ORM/parameterized queries  \n- **Validate inputs** (e.g., username/password formats)  \n- **Update dependencies** (SQLAlchemy, FastAPI)  \n- **Deadline**: Patch within 3 days (VUL-c321-2026-10-06)",
  "session_id": "s0932-demo-session",
  "memory_used": {
    "short_term_count": 12,
    "short_term_messages": [
      {
        "role": "user",
        "content": "We mostly use Python and FastAPI for our internal tools"
      },
      {
        "role": "assistant",
        "content": "Hello, test! Great to know you're using Python and FastAPI. For a small bank, focus on securing internal tools with practices like regular dependency updates, input validation, and secure authentication. Would you like guidance on specific areas like CI/CD pipelines, dependency checks, or secure FastAPI configurations?"
      },
      {
        "role": "user",
        "content": "The biggest concern is the SQL injection in the login service for our bank"
      },
      {
        "role": "assistant",
        "content": "To mitigate SQL injection in your login service:  \n1. **Use ORM (e.g., SQLAlchemy)** to abstract SQL queries, automatically escaping inputs.  \n2. **Parameterize all queries** (e.g., `cursor.execute(\"SELECT * FROM users WHERE username = %s\", (username,))`).  \n3. **Validate and sanitize inputs** (e.g., check for valid username/password formats).  \n4. **Avoid raw SQL** unless absolutely necessary, and even then, use parameterized queries.  \n\nWould you like a sample FastAPI/SQLAlchemy implementation for this?"
      },
      {
        "role": "user",
        "content": "The login service is owned by the software team manager, Dolittle"
      },
      {
        "role": "assistant",
        "content": "Would you like to escalate this to Dolittle (the login service owner) or need technical steps to fix SQL injection in their service? I can help draft a risk report or provide code examples."
      },
      {
        "role": "user",
        "content": "Please keep your answers short, I prefer bullet points answers"
      },
      {
```



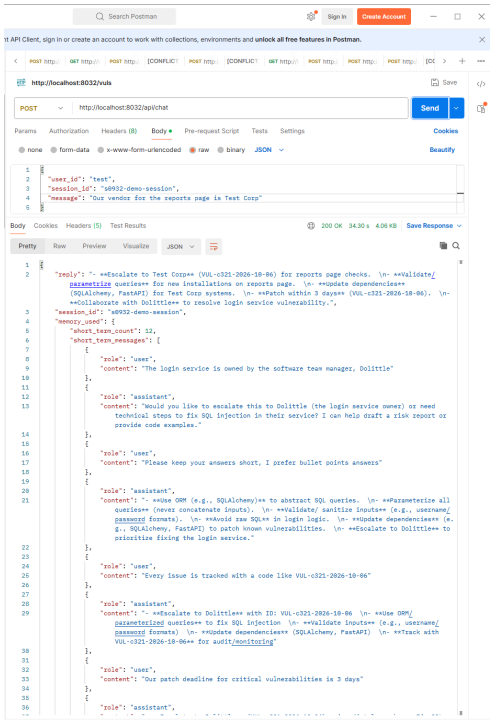
POST http://localhost:8032/api/chat

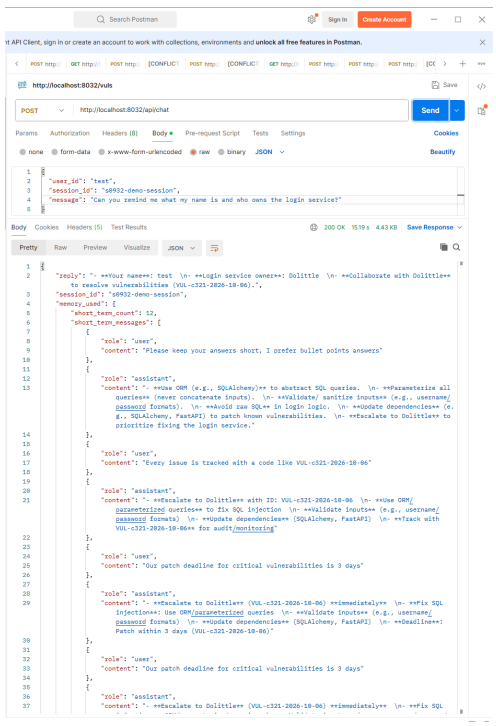
Body (raw JSON)

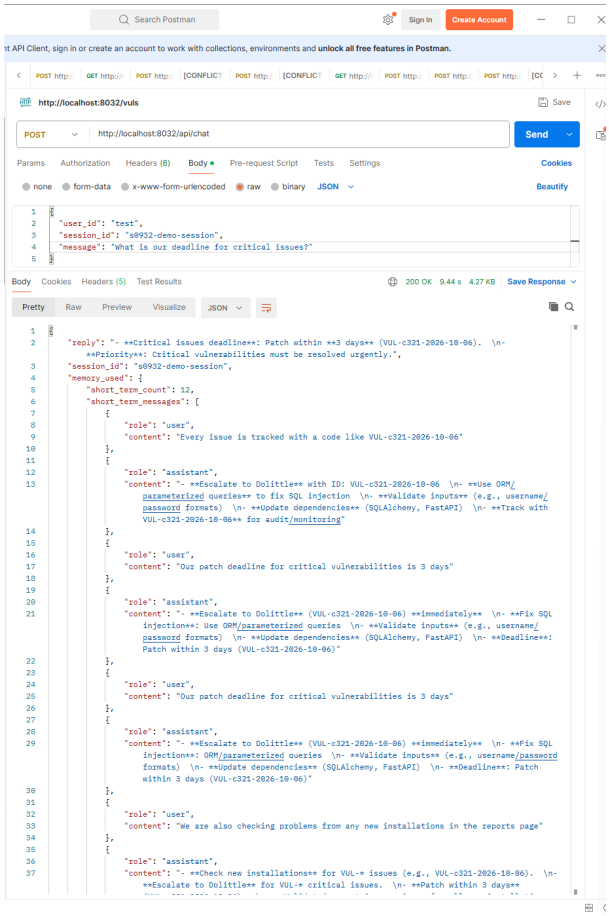
```
{
  "user_id": "test",
  "session_id": "s0932-demo-session",
  "message": "We are also checking problems from any new installations in the reports page"
}
```

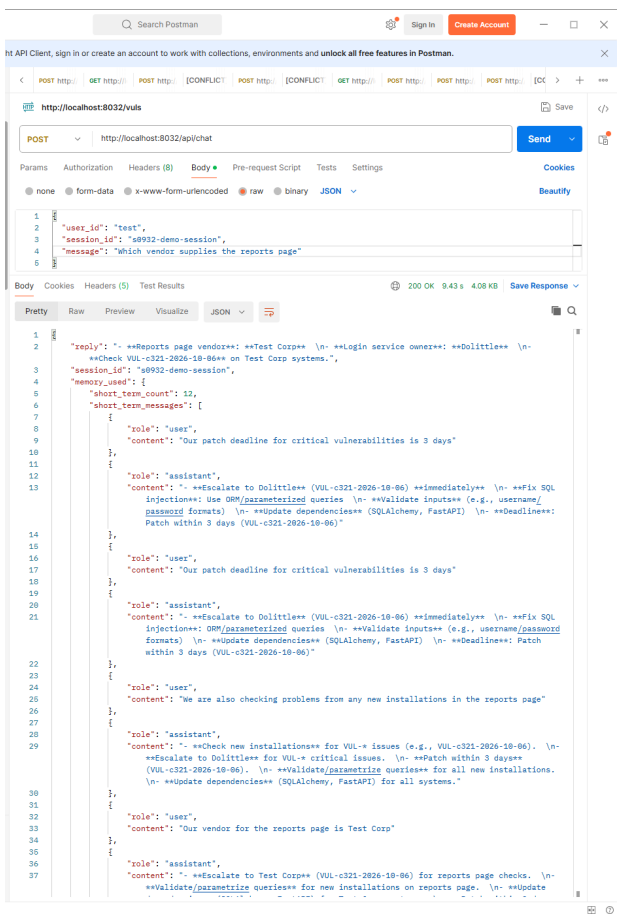
Response (200 OK, 22.88 s, 4.27 KB)

```
{
  "reply": "- **Check new installations** for VUL-* issues (e.g., VUL-c321-2026-10-06).  \n- **Escalate to Dolittle** for VUL-* critical issues.  \n- **Patch within 3 days** (VUL-c321-2026-10-06).  \n- **Validate/parametrize queries** for all new installations.  \n- **Update dependencies** (SQLAlchemy, FastAPI) for all systems.",
  "session_id": "s0932-demo-session",
  "memory_used": {
    "short_term_count": 12,
    "short_term_messages": [
      {
        "role": "user",
        "content": "The biggest concern is the SQL injection in the login service for our bank"
      },
      {
        "role": "assistant",
        "content": "To mitigate SQL injection in your login service:  \n1. **Use ORM (e.g., SQLAlchemy)** to abstract SQL queries, automatically escaping inputs.  \n2. **Parameterize all queries** (e.g., `cursor.execute(\"SELECT * FROM users WHERE username = %s\", (username,))`).  \n3. **Validate and sanitize inputs** (e.g., check for valid username/password formats).  \n4. **Avoid raw SQL** unless absolutely necessary, and even then, use parameterized queries.  \n\nWould you like a sample FastAPI/SQLAlchemy implementation for this?"
      },
      {
        "role": "user",
        "content": "The login service is owned by the software team manager, Dolittle"
      },
      {
        "role": "assistant",
        "content": "Would you like to escalate this to Dolittle (the login service owner) or need technical steps to fix SQL injection in their service? I can help draft a risk report or provide code examples."
      },
      {
        "role": "user",
        "content": "Please keep your answers short, I prefer bullet points answers"
      },
      {
        "role": "assistant",
        "content": "- **Use ORM (e.g., SQLAlchemy)** to abstract SQL queries.  \n- **Parameterize all queries** (never concatenate inputs).  \n- **Validate/ sanitize inputs** (e.g., username/password formats).  \n- **Avoid raw SQL** in login logic.  \n- **Update dependencies** (e.g., SQLAlchemy, FastAPI) to patch known vulnerabilities.  \n- **Escalate to Dolittle** to prioritize fixing the login service."
      },
      {
        "role": "user",
        "content": "Every issue is tracked with a code like VUL-c321-2026-10-06"
      },
      {
```

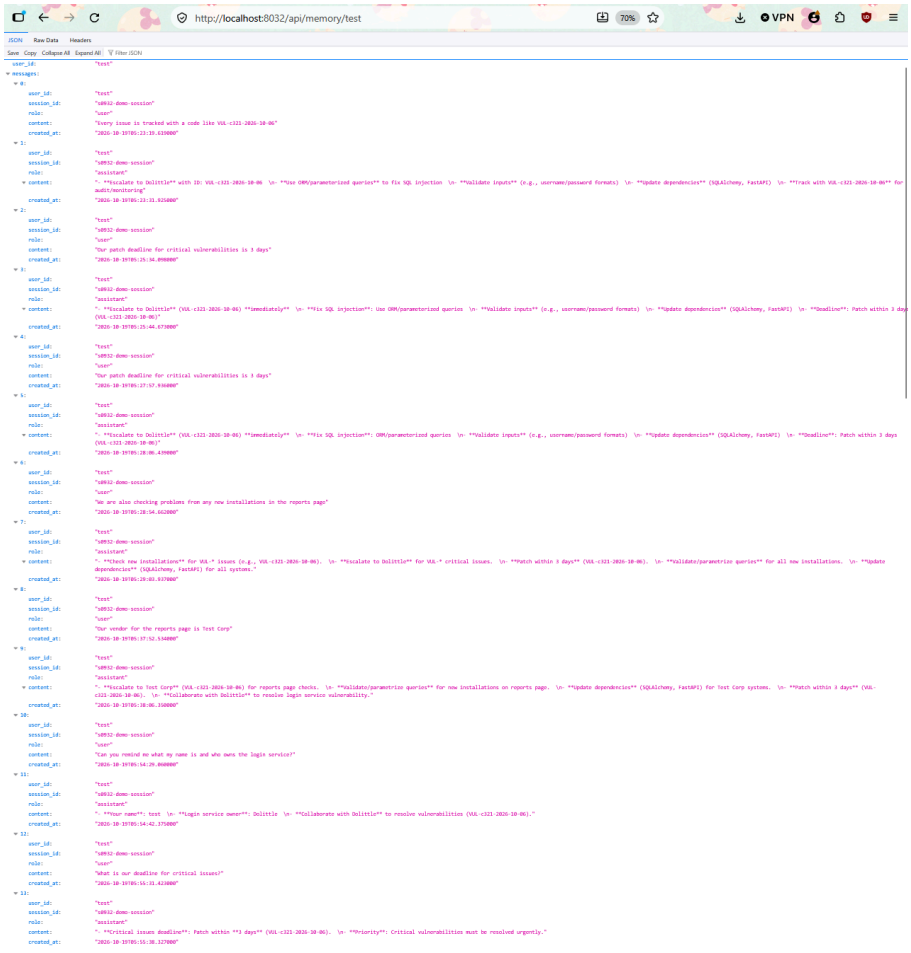


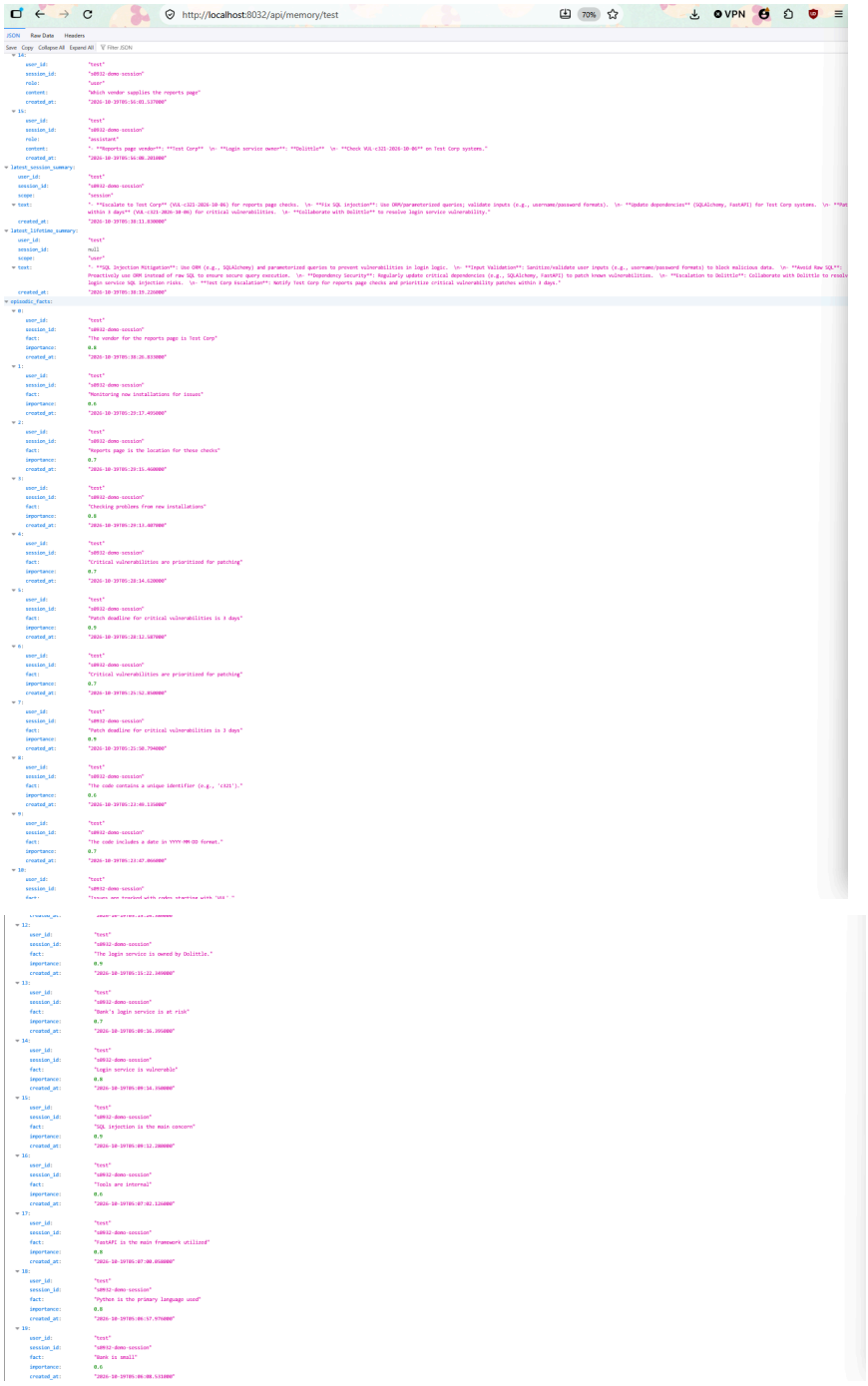




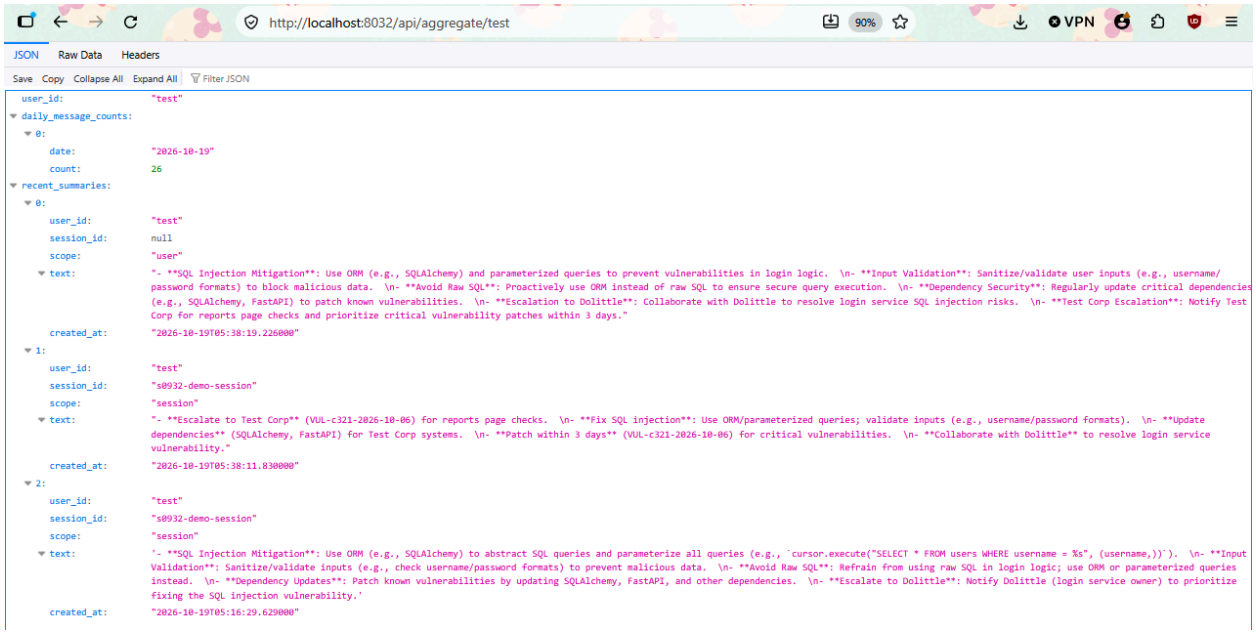


the JSON result of GET /api/memory/{user_id}

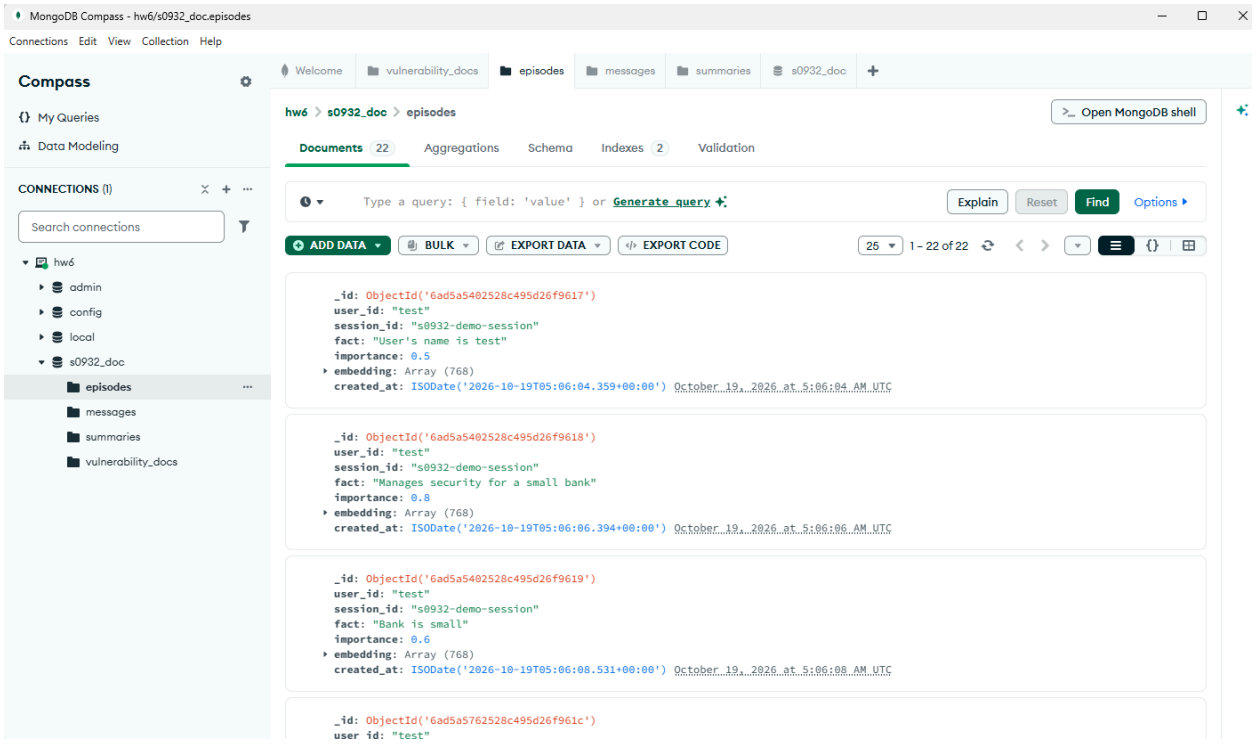

http://localhost:8032/api/memory/test
70%
VPN
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
user_id: "test"
messages:
0:
user_id: "test"
session_id: "s0932-demo-session"
role: "user"
content: "Every issue is tracked with a code like VUL-c321-2026-10-06"
created_at: "2026-10-19T05:23:19.639000"
1:
user_id: "test"
session_id: "s0932-demo-session"
role: "assistant"
content: ". **Escalate to Dolittle** with ID: VUL-c321-2026-10-06 \n- **Use ORM/parameterized queries** to fix SQL injection \n- **Validate inputs** (e.g., username/password formats) \n- **Update dependencies** (SQLAlchemy, FastAPI) \n- **Track with VUL-c321-2026-10-06** for audit/monitoring"
created_at: "2026-10-19T05:23:31.825000"
2:
user_id: "test"
session_id: "s0932-demo-session"
role: "user"
content: "Our patch deadline for critical vulnerabilities is 3 days"
created_at: "2026-10-19T05:25:54.098000"
3:
user_id: "test"
session_id: "s0932-demo-session"
role: "assistant"
content: ". **Escalate to Dolittle** (VUL-c321-2026-10-06) **immediately** \n- **Fix SQL injection**: Use ORM/parameterized queries \n- **Validate inputs** (e.g., username/password formats) \n- **Update dependencies** (SQLAlchemy, FastAPI) \n- **Deadline**: Patch within 3 days (VUL-c321-2026-10-06)"
created_at: "2026-10-19T05:25:44.673000"
4:
user_id: "test"
session_id: "s0932-demo-session"
role: "user"
content: "Our patch deadline for critical vulnerabilities is 3 days"
created_at: "2026-10-19T05:27:57.936000"
5:
user_id: "test"
session_id: "s0932-demo-session"
role: "assistant"
content: ". **Escalate to Dolittle** (VUL-c321-2026-10-06) **immediately** \n- **Fix SQL injection**: ORM/parameterized queries \n- **Validate inputs** (e.g., username/password formats) \n- **Update dependencies** (SQLAlchemy, FastAPI) \n- **Deadline**: Patch within 3 days (VUL-c321-2026-10-06)"
created_at: "2026-10-19T05:28:06.439000"
6:
user_id: "test"
session_id: "s0932-demo-session"
role: "user"
content: "We are also checking problems from any new installations in the reports page"
created_at: "2026-10-19T05:28:54.662000"
7:
user_id: "test"
session_id: "s0932-demo-session"
role: "assistant"
content: ". **Track new installations** for VUL-* issues (e.g., VUL-c321-2026-10-06). \n- **Escalate to Dolittle** for VUL-* critical issues. \n- **Patch within 3 days** (VUL-c321-2026-10-06). \n- **Validate/parametrize queries** for all new installations. \n- **Update dependencies** (SQLAlchemy, FastAPI) for all systems."
created_at: "2026-10-19T05:29:03.937000"
8:
user_id: "test"
session_id: "s0932-demo-session"
role: "user"
content: "Our vendor for the reports page is Test Corp"
created_at: "2026-10-19T05:37:52.534000"
9:
user_id: "test"
session_id: "s0932-demo-session"
role: "assistant"
content: ". **Escalate to Test Corp** (VUL-c321-2026-10-06) for reports page checks. \n- **Validate/parametrize queries** for new installations on reports page. \n- **Update dependencies** (SQLAlchemy, FastAPI) for Test Corp systems. \n- **Patch within 3 days** (VUL-c321-2026-10-06). \n- **Collaborate with Dolittle** to resolve login service vulnerability."
created_at: "2026-10-19T05:38:06.350000"
10:
user_id: "test"
session_id: "s0932-demo-session"
role: "user"
content: "Can you remind me what my name is and who owns the login service?"
created_at: "2026-10-19T05:54:29.868000"
11:
user_id: "test"
session_id: "s0932-demo-session"
role: "assistant"
content: ". **Your name**: test \n- **Login service owner**: Dolittle \n- **Collaborate with Dolittle** to resolve vulnerabilities (VUL-c321-2026-10-06)."
created_at: "2026-10-19T05:54:42.375000"
12:
user_id: "test"
session_id: "s0932-demo-session"
role: "user"
content: "What is our deadline for critical issues?"
created_at: "2026-10-19T05:55:31.423000"
13:
user_id: "test"
session_id: "s0932-demo-session"
role: "assistant"
content: ". **Critical issues deadline**: Patch within **3 days** (VUL-c321-2026-10-06). \n- **Priority**: Critical vulnerabilities must be resolved urgently."
created_at: "2026-10-19T05:55:38.327000"

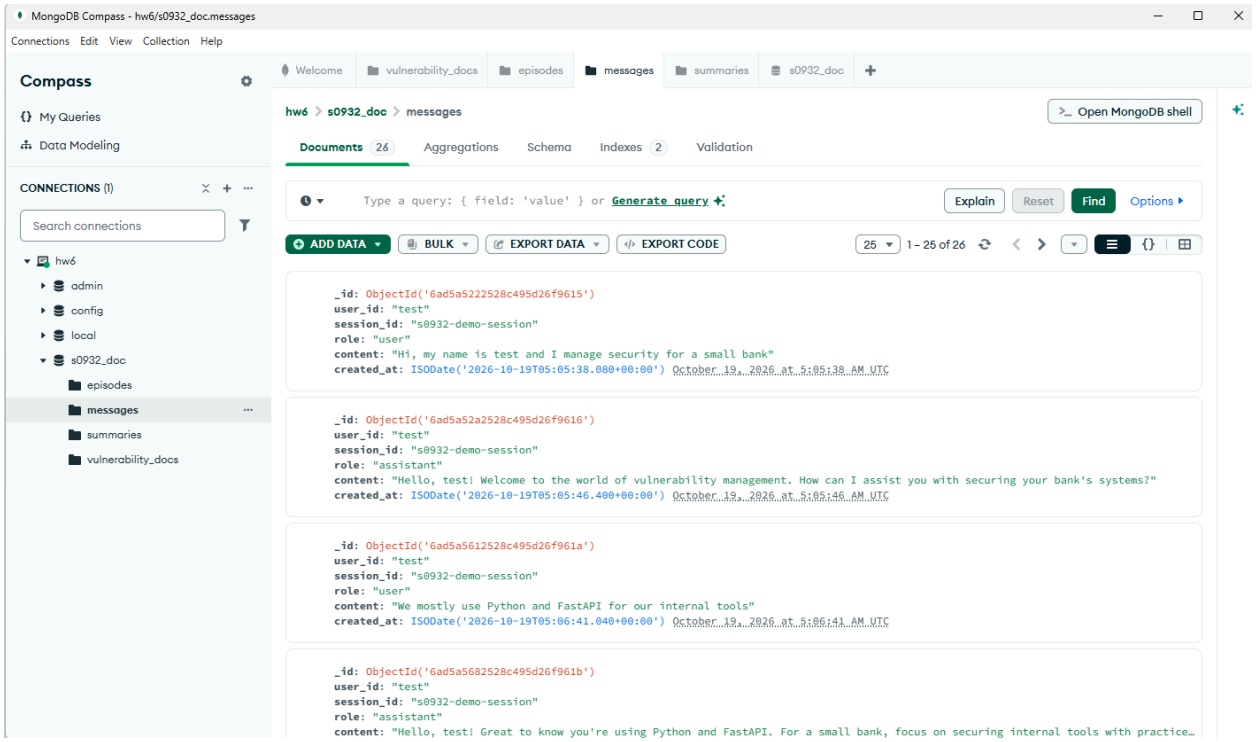


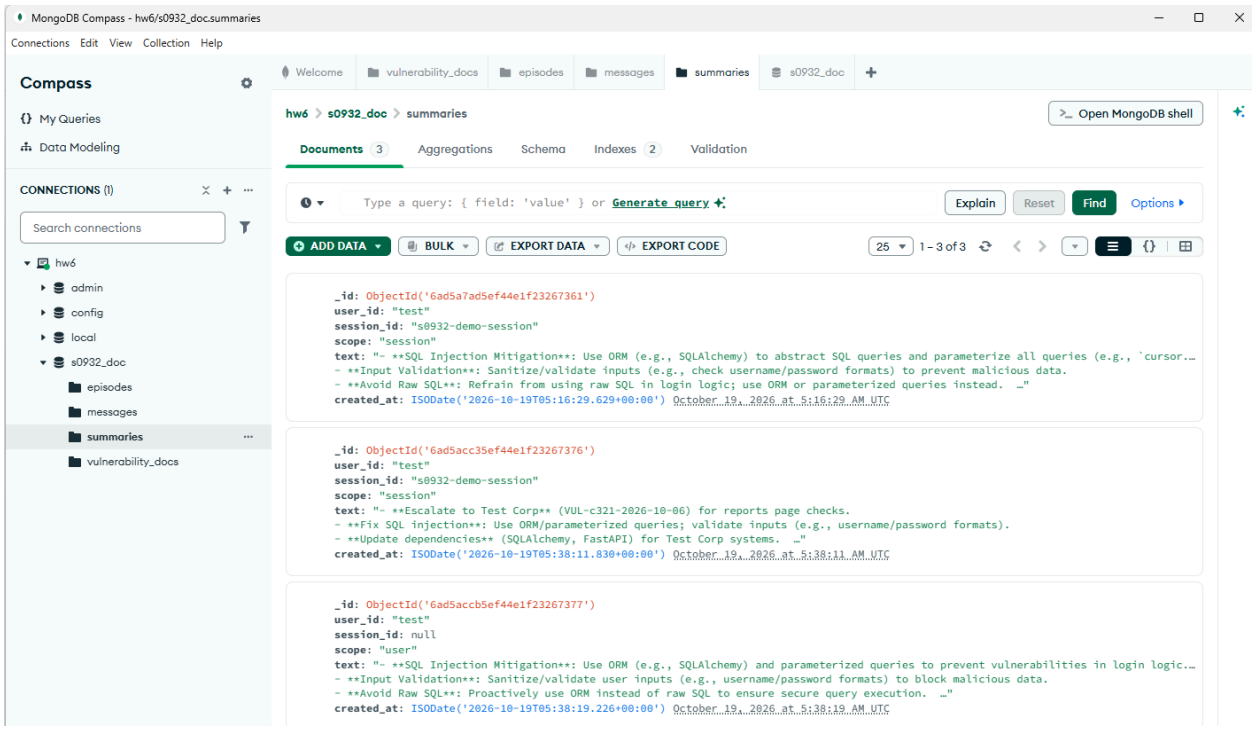
the JSON result of GET /api/aggregate/{user_id}



MongoDB Compass showing the messages, summaries, and episodes collections with documents







and your code for the three endpoints plus the memory logic functions (episode extraction, summarization trigger, embedding + retrieval, prompt assembly).

Episode extraction

```python
def extract_facts(user_message):
    prompt = ("Extract up to 3 short facts worth remembering from the user message below."
        'Reply with ONLY a JSON list like [{"fact": " ... ", "importance": 0.7}].'
        "importance is a number from 0 to 1."
        "If nothing is worth remembering reply with [].\n\n"
        "User message: " + user_message)
    text, _, _= llm_adapter.chat([{"role": "user", "content": prompt}])
    start = text.find("[")
    end = text.rfind("]")
    if start == -1 or end == -1:
        return []
    try:
        items = json.loads(text[start:end + 1])
    except json.JSONDecodeError:
        return []

    facts = []
    for item in items[:3]:
        if not isinstance(item, dict) or not item.get("fact"):
            continue
        try:
            importance = float(item.get("importance", 0.5))
        except (TypeError, ValueError):
            importance = 0.5
        importance = max(0.0, min(1.0, importance))
        facts.append({"fact": str(item["fact"])[:200], "importance": importance})
    return facts
```

Summarization trigger

```python
def maybe_summarize(user_id, session_id):
    user_msg_count = get_messages_col().count_documents(
    {"user_id": user_id, "session_id": session_id, "role": "user"})
    if user_msg_count == 0 or user_msg_count % SUMMARIZE_EVERY_USER_MSGS != 0:
        return False

    recent = get_short_term(user_id, session_id, SUMMARIZE_EVERY_USER_MSGS * 2)
    transcript = "\n".join(m["role"] + ": " + m["content"] for m in recent)
    prompt = ("Summarize this conversation into 3-5 concise bullet points."
        "Keep names, numbers and decisions. \n\n" + transcript)
    text, _, _= llm_adapter.chat([{"role": "user", "content": prompt}])
    save_summary(user_id, session_id, "session", text)
    session_summary_count = get_summaries_col().count_documents(
        {"user_id": user_id, "scope": "session"})
    if session_summary_count % CONDENSE_EVERY_N_SUMMARIES == 0:
        condense_lifetime(user_id)
    return True

def condense_lifetime(user_id):
    rows = list(get_summaries_col()
        .find({"user_id": user_id, "scope": "session"}, {"_id": 0, "text": 1})
        .sort([("created_at", -1), ("_id", -1)])
        .limit(CONDENSE_EVERY_N_SUMMARIES))
    rows.reverse ()
    old_lifetime = get_latest_summary(user_id, "user")
    parts = []
    if old_lifetime:
        parts.append("Previous lifetime summary:\n" + old_lifetime)
    for r in rows:
        parts.append("Session summary: \n" + r["text"])
    prompt = ("Merge the summaries below into one short lifetime summary"
        "(max 6 bullet points) about this user.\n\n" + "\n\n".join(parts))
    text, _, _= llm_adapter.chat([{"role": "user", "content": prompt}])
    save_summary(user_id, None, "user", text)
```

Embedding + Retrieval

```python
def cosine_similarity(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    len_a = math.sqrt(sum(x * x for x in a))
    len_b = math.sqrt(sum(y * y for y in b))
    if len_a == 0 or len_b == 0:
        return 0.0
    return dot / (len_a * len_b)

def retrieve_episodes(user_id, message, k=TOP_K):
    episodes = list(get_episodes_col().find({"user_id": user_id}))
    if not episodes:
        return []
    query_vec = llm_adapter.embed (message)
    scored = []
    for ep in episodes:
        score = cosine_similarity(query_vec, ep["embedding"])
        scored.append({"fact": ep["fact"], "importance": ep["importance"],
            "score": round(score, 4)})
    scored.sort(key=lambda e: e["score"], reverse=True)# best match first
    return scored[:k]
```

Prompt assembly

```python
def build_prompt(session_summary, lifetime_summary, window, message, facts):
    system_text = SYSTEM_PRIMER
    if lifetime_summary:
        system_text += "\n\nLifetime summary of this user: \n" + lifetime_summary
    if session_summary:
        system_text += "\n\nSummary of this session so far: \n" + session_summary
    messages = [{"role": "system", "content": system_text}]
    messages.extend (window)
    final = message
    if facts:
        final += "\n\n(Relevant remembered facts: \n"
        final += "\n".join("- " + f["fact"] for f in facts) + ")"
    messages.append({"role": "user", "content": final})
    return messages
```

Part 3 - What Memory Costs

1. Log memory usage

For each /api/chat request, record:

• Prompt and completion tokens

• Tokens used by the system primer, long-term summary, short-term window, and episodic facts

• Request latency

```python
def log_usage(row):
    os.makedirs(RAW_DIR, exist_ok=True)
    is_new_file = not os.path.exists(LOG_FILE)
    row["timestamp"] = datetime.now(timezone.utc).isoformat()
    with open(LOG_FILE, "a", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=FIELDS)
        if is_new_file:
            writer.writeheader()
        writer.writerow(row)
```

```python
@router.post("/chat")
def chat(payload: ChatRequest):
    start=time.perf_counter()
    user_id = payload.user_id
    session_id = payload.session_id or str(uuid.uuid4())
    n=payload.short_term_n or mem.SHORT_TERM_N
    policy=payload.policy or "none"
    try:
        mem.save_message(user_id, session_id, "user", payload.message)
        window = mem.get_short_term(user_id, session_id, SHORT_TERM_N + 1)[ :- 1]
        session_summary = mem.get_latest_summary(user_id, "session", session_id)
        lifetime_summary = mem.get_latest_summary(user_id, "user")
        facts = mem.retrieve_episodes(user_id, payload.message)
        prompt = mem.build_prompt(session_summary, lifetime_summary,
        window, payload.message, facts)
        reply, prompt_tokens, completion_tokens = llm_adapter.chat(prompt)
        mem.save_message(user_id, session_id, "assistant", reply)
        summary_created = mem.maybe_summarize(user_id, session_id)
        mem.save_episodes(user_id, session_id, mem.extract_facts(payload.message))

    except RuntimeError as e:
        raise HTTPException(status_code=502, detail=str(e))
    except PyMongoError:
        raise HTTPException(status_code=500, detail="Database error in chat")
    summary_text = (lifetime_summary or "") + (session_summary or "")
    metrics = {
        "prompt_tokens": prompt_tokens,
        "completion_tokens": completion_tokens,
        "system_primer_tokens_est": mem.estimate_tokens(mem.SYSTEM_PRIMER),
        "long_term_summary_tokens_est": mem.estimate_tokens(summary_text),
        "short_term_tokens_est": sum(mem.estimate_tokens(m["content"]) for m in window),
        "episodic_tokens_est": mem.estimate_tokens("\n".join(f["fact"] for f in facts)),
        "latency_ms": round((time.perf_counter() - start) * 1000),
    }
    # save one row chat_usage.csv
    log.log_usage({ **metrics,
        "run_label": payload.run_label or "manual",
        "policy": policy, "short_term_n": n,
        "user_id": user_id, "session_id": session_id})
    return {
        "reply": reply,
        "session_id": session_id,
        "memory_used": {
            "short_term_count": len(window),
            "short_term_messages": window,
            "session_summary": session_summary,
            "lifetime_summary": lifetime_summary,
            "episodic_facts": facts,
            "new_summary_created": summary_created,
        },
        "metrics":metrics,
    }
```

Save the raw results as CSV or JSON.




```
timestamp,run_label,policy,short_term_n,user_id,session_id,prompt_tokens,completion_tokens,system_primer_tokens_est,long_te
2026-10-19T07:03:31.040113+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,61,134,33,0,0,0,14647
2026-10-19T07:03:44.319939+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,150,197,33,0,76,18,11253
2026-10-19T07:04:06.662753+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,200,200,33,0,118,25,20328
2026-10-19T07:04:24.390412+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,266,224,33,0,194,25,15699
2026-10-19T07:04:49.253841+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,328,239,33,0,264,25,22825
2026-10-19T07:05:03.596452+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,551,267,33,136,333,41,12311
2026-10-19T07:05:16.743955+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,606,389,33,136,379,34,11101
2026-10-19T07:05:39.363416+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,606,331,33,136,367,70,20597
2026-10-19T07:05:58.542459+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,634,241,33,136,404,61,17152
2026-10-19T07:06:36.452944+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,627,260,33,136,406,30,35863
2026-10-19T07:06:56.244391+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,883,301,33,433,429,56,17757
2026-10-19T07:07:17.311413+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,873,386,33,433,445,58,19035
2026-10-19T07:07:39.650554+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,916,352,33,433,495,65,20311
2026-10-19T07:07:58.585765+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,919,259,33,433,517,55,16917
2026-10-19T07:08:31.728337+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,934,378,33,433,543,55,31110
2026-10-19T07:08:54.915474+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,1019,507,33,462,579,68,21159
2026-10-19T07:09:20.646324+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,1010,525,33,462,598,48,23701
2026-10-19T07:09:39.914584+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,1075,325,33,462,666,65,17227
2026-10-19T07:10:04.172510+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,1095,363,33,462,698,68,22223
2026-10-19T07:10:50.342898+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,1118,321,33,462,746,45,44154
2026-10-19T07:11:12.933718+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,1125,453,33,446,768,57,20547
2026-10-19T07:11:34.916915+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,1151,397,33,446,774,69,19956
2026-10-19T07:11:55.587983+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,1184,335,33,446,794,74,18643
2026-10-19T07:12:21.012000+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,1178,625,33,446,767,71,23384
2026-10-19T07:12:49.814132+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,1203,433,33,446,802,57,26773
2026-10-19T07:13:10.936861+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,1197,396,33,436,795,61,19099
2026-10-19T07:13:35.843921+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,1198,304,33,436,794,64,22875
2026-10-19T07:13:55.610844+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,1190,214,33,436,809,67,17747
2026-10-19T07:14:12.611238+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,1150,576,33,436,789,61,14981
2026-10-19T07:14:43.943938+00:00,N4,none,4,exp_N4_1792393394,N4-session-1792393394,1335,273,33,436,1032,62,29285
2026-10-19T07:14:56.021061+00:00,N4_probe,none,4,exp_N4_1792393394,N4-session-1792393394,1260,221,33,475,928,33,9967
2026-10-19T07:15:06.650454+00:00,N4_probe,none,4,exp_N4_1792393394,N4-session-1792393394,1176,176,33,475,826,43,8593
2026-10-19T07:15:15.538030+00:00,N4_probe,none,4,exp_N4_1792393394,N4-session-1792393394,1118,234,33,475,739,51,6862
2026-10-19T07:15:27.347920+00:00,N4_probe,none,4,exp_N4_1792393394,N4-session-1792393394,1046,276,33,475,643,43,9745
```

2. Create memory probes

Before testing, create memory_probes.yaml with five fixed questions, expected answer criteria, and a 1–5 scoring rubric.

reports > hw06 > ! memory_probes.yaml

```yaml
probes :
  - id: P1
    question: "What is my name and which company do I work for?"
    expected_answer: "Alice, Northwind Bank"
    expected_keywords: ["alice","northwind"]
    planted_at_turn: 1

  - id: P2
    question: "Who owns the login service?"
    expected_answer: "Marcus Reed"
    expected_keywords: ["marcus"]
    planted_at_turn: 10

  - id: P3
    question: "What is our patch deadline for critical issues?"
    expected_answer: "7 days"
    expected_keywords: ["7|seven"]
    planted_at_turn: 18

  - id: P4
    question: "Which vendor supplies the reports page?"
    expected_answer: "Acme Corp"
    expected_keywords: ["acme"]
    planted_at_turn: 25

  - id: P5
    question: "What answer format did I ask you to use?"
    expected_answer: "Bullet points"
    expected_keywords: ["bullet"]
    planted_at_turn: 29
```

3. Compare short-term memory sizes

Run the same 30-message conversation in fresh sessions with:

• SHORT_TERM_N = 4

• SHORT_TERM_N = 12

• SHORT_TERM_N = 30

```python
RUNS = [
    ("N4", 4, "none"),
    ("N12", 12, "none"),
    ("N30", 30, "none"),
    ("COMPRESS", 12, "compress"),
]
```

Ask the same five probe questions after each run. Report the mean prompt tokens per turn, mean latency, and mean probe-quality score.

```
for p in probes: #5 probe questions are asked in the same session after the run
    data = send(user_id, session_id, p["question"], n, policy, label + "_probe")
    score, matched = score_answer(data["reply"], p["expected_keywords"])
    probe_results.append({"run": label, "probe_id": p["id"], "question": p["question"],
        "expected": p["expected_answer"], "answer": data["reply"],
        "matched_keywords": matched, "auto_score": score})
    log(f"{label} probe {p['id']} score={score}")

summary={
    "setting": label, "short_term_n": n, "policy": policy,
    "mean_prompt_tokens": round(statistics.mean(r["prompt_tokens"] for r in turn_rows), 1),
    "mean_completion_tokens": round(statistics.mean(r["completion_tokens"] for r in turn_rows), 1),
    "mean_latency_ms": round(statistics.mean(r["latency_ms"] for r in turn_rows)),
    "probe_quality": round(statistics.mean(r["auto_score"] for r in probe_results), 2),
}
return turn_rows, probe_results, summary
```

4. Test one memory-saving policy

Implement one eviction or compression policy. Repeat the conversation and probes, then report token savings and any change in probe quality.

```
160    def compress_window(window) :
161        result = []
162        cutoff = len(window) - 4
163        for i, m in enumerate(window):
164            content = m["content"]
165            if m["role"] == "assistant" and i < cutoff and len(content) > 150:
166                content = content[:150] + " .."
167            result.append({"role": m["role"], "content": content})
168        return result
```

```
comp = next(s for s in summaries if s["setting"] == "COMPRESS")
saved = (base["mean_prompt_tokens"] - comp["mean_prompt_tokens"]) / base["mean_prompt_tokens"] * 100
lines += ["", f"Compression vs N12: token savings = {saved :.1f}%,"
    f"probe quality change = {comp[ 'probe_quality'] - base['probe_quality']:+.2f}",
    "", f"Cost of {CONVERSATIONS} x {TURNS}-turn conversations"
    f"(input ${INPUT_PRICE_PER_M}/M, output ${OUTPUT_PRICE_PER_M}/M):",
    "| Setting | Probe quality | Input tokens | Output tokens | Cost (USD) | Role |",
    "| --- | --- | --- | --- | --- | --- |"]
for r in cost_rows:
    lines.append(f"| {r['setting']} | {r['probe_quality']} | {r['input_tokens']:,} |"
        f"{r[ 'output_tokens']:,} | ${r['cost_usd']} | {r['role']} |")
```

```
PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw06> python .\part3.py
00:40:56 N30 probe P5 score=5
00:40:56  run COMPRESS: SHORT_TERM_N=12, policy=compress
00:41:14 COMPRESS turn 1/30 prompt=61completion=111 latency=14348ms
00:41:28 COMPRESS turn 2/30 prompt=132completion=223 latency=11594ms
00:41:51 COMPRESS turn 3/30 prompt=181completion=266 latency=21435ms
00:42:10 COMPRESS turn 4/30 prompt=244completion=271 latency=16516ms
00:42:38 COMPRESS turn 5/30 prompt=316completion=237 latency=26314ms
00:42:54 COMPRESS turn 6/30 prompt=529completion=380 latency=14126ms
00:43:06 COMPRESS turn 7/30 prompt=584completion=276 latency=9883ms
00:43:30 COMPRESS turn 8/30 prompt=595completion=349 latency=21559ms
00:43:49 COMPRESS turn 9/30 prompt=622completion=247 latency=17245ms
00:44:28 COMPRESS turn 10/30 prompt=618completion=371 latency=36374ms
00:44:47 COMPRESS turn 11/30 prompt=887completion=322 latency=17756ms
00:45:08 COMPRESS turn 12/30 prompt=898completion=354 latency=18816ms
00:45:32 COMPRESS turn 13/30 prompt=971completion=422 latency=21280ms
00:45:53 COMPRESS turn 14/30 prompt=988completion=399 latency=18990ms
00:46:25 COMPRESS turn 15/30 prompt=1017completion=324 latency=29909ms
00:46:46 COMPRESS turn 16/30 prompt=1100completion=382 latency=19670ms
00:47:11 COMPRESS turn 17/30 prompt=1101completion=464 latency=22923ms
00:47:35 COMPRESS turn 18/30 prompt=1136completion=643 latency=21924ms
00:48:00 COMPRESS turn 19/30 prompt=1107completion=407 latency=23189ms
00:48:46 COMPRESS turn 20/30 prompt=1111completion=430 latency=43852ms
00:49:09 COMPRESS turn 21/30 prompt=1147completion=411 latency=20480ms
00:49:32 COMPRESS turn 22/30 prompt=1168completion=465 latency=20870ms
00:49:53 COMPRESS turn 23/30 prompt=1174completion=377 latency=19510ms
00:50:17 COMPRESS turn 24/30 prompt=1167completion=527 latency=21965ms
00:50:44 COMPRESS turn 25/30 prompt=1207completion=303 latency=24254ms
00:51:04 COMPRESS turn 26/30 prompt=1210completion=317 latency=18106ms
00:51:30 COMPRESS turn 27/30 prompt=1199completion=418 latency=24702ms
00:51:51 COMPRESS turn 28/30 prompt=1218completion=236 latency=18209ms
00:52:05 COMPRESS turn 29/30 prompt=1201completion=308 latency=12584ms
00:52:51 COMPRESS turn 30/30 prompt=1166completion=169 latency=43792ms
00:53:03 COMPRESS probe P1 score=3
00:53:14 COMPRESS probe P2 score=5
00:53:22 COMPRESS probe P3 score=5
00:53:31 COMPRESS probe P4 score=5
00:53:51 COMPRESS probe P5 score=5
00:53:51
| Setting | Mean prompt tokens/turn | Mean latency (ms) | Probe quality (1-5) |
| --- | --- | --- | --- |
| N4 | 859.4 | 20955 | 4.6 |
| N12 | 825.0 | 20389 | 4.6 |
| N30 | 886.8 | 21343 | 4.2 |
| COMPRESS | 868.5 | 21739 | 4.6 |

Compression vs N12: token savings = -5.3%,probe quality change = +0.00

Cost of 10000 x 30-turn conversations(input $1.0/M, output $5.0/M):
| Setting | Probe quality | Input tokens | Output tokens | Cost (USD) | Role |
| --- | --- | --- | --- | --- | --- |
| N4 | 4.6 | 257,820,000 |102,060,000 | $768.12 |  |
| N12 | 4.6 | 247,500,000 |101,850,000 | $756.75 | BEST |
| N30 | 4.2 | 266,040,000 |99,960,000 | $765.84 |  |
| COMPRESS | 4.6 | 260,550,000 |104,100,000 | $781.05 | WORST |
```

5. Estimate memory cost

Using input pricing of $1 per million tokens and output pricing of $5 per million tokens, estimate the cost of 10,000 30-turn conversations under your best and worst settings.

```python
#pricing
INPUT_PRICE_PER_M = 1.0 # dollars per million tokens
OUTPUT_PRICE_PER_M = 5.0
CONVERSATIONS = 10000
TURNS = 30
```

```python
#cost of 10,000 30-turn conversations
cost_rows = []
for s in summaries:
    input_tokens = s["mean_prompt_tokens"] * TURNS * CONVERSATIONS
    output_tokens = s["mean_completion_tokens"] * TURNS * CONVERSATIONS
    cost = input_tokens / 1e6 * INPUT_PRICE_PER_M + output_tokens / 1e6 * OUTPUT_PRICE_PER_M
    cost_rows.append({"setting": s["setting"], "probe_quality": s["probe_quality"],
        "input_tokens": int(input_tokens), "output_tokens": int(output_tokens),
        "cost_usd": round(cost, 2)})
top_quality = max(r["probe_quality"] for r in cost_rows)
good_enough = [r for r in cost_rows if r["probe_quality"] >= top_quality - 0.5]
best = min(good_enough, key=lambda r: r["cost_usd"])["setting"]
worst = max(cost_rows, key=lambda r: r["cost_usd"])["setting"]
for r in cost_rows:
    r["role"] ="BEST" if r["setting"] == best else ("WORST" if r["setting"] == worst else "")
write_csv(os.path.join(RAW_DIR, "cost_estimate.csv"), cost_rows)
```

No paid hosted-model run is required.

Explain how you selected the best and worst settings.

I had best to be where the cheapest setting is within 0.5 of the top quality score. This worst setting is where I picked the most expensive setting.

memory-cost table

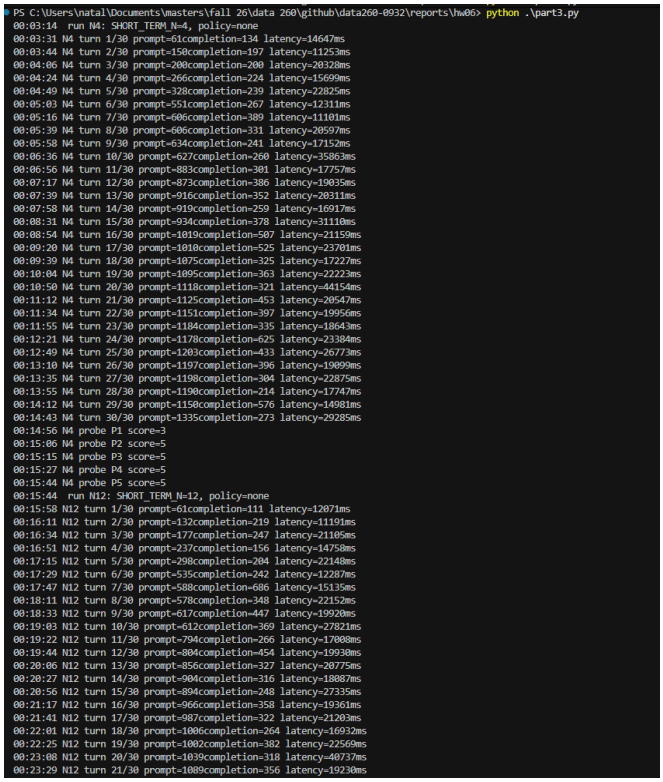
reports > hw06 > raw > part3_tables.md

```
| Setting | Mean prompt tokens/turn | Mean latency (ms) | Probe quality (1-5) |
| --- | --- | --- | --- |
| N4 | 859.4 | 20955 | 4.6 |
| N12 | 825.0 | 20389 | 4.6 |
| N30 | 886.8 | 21343 | 4.2 |
| COMPRESS | 868.5 | 21739 | 4.6 |

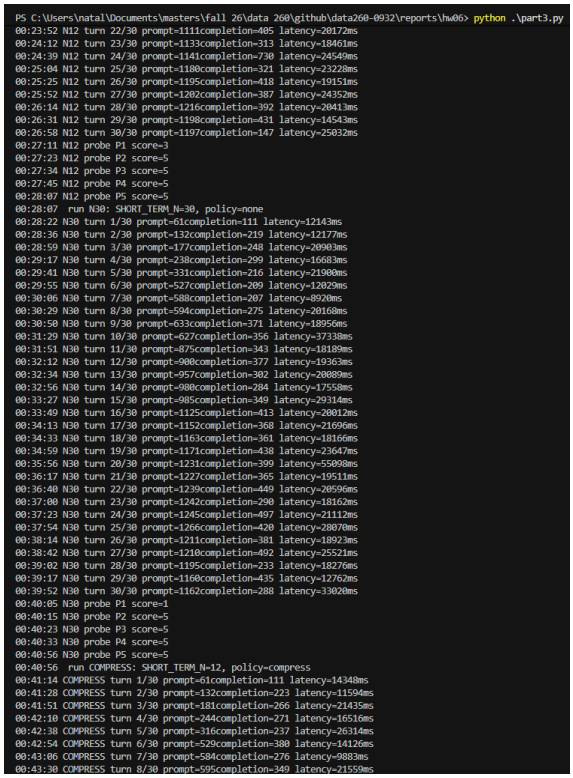
Compression vs N12: token savings = -5.3%,probe quality change = +0.00

Cost of 10000 x 30-turn conversations(input $1.0/M, output $5.0/M):
| Setting | Probe quality | Input tokens | Output tokens | Cost (USD) | Role |
| --- | --- | --- | --- | --- | --- |
| N4 | 4.6 | 257,820,000 |102,060,000 | $768.12 |  |
| N12 | 4.6 | 247,500,000 |101,850,000 | $756.75 | BEST |
| N30 | 4.2 | 266,040,000 |99,960,000 | $765.84 |  |
| COMPRESS | 4.6 | 260,550,000 |104,100,000 | $781.05 | WORST |
```

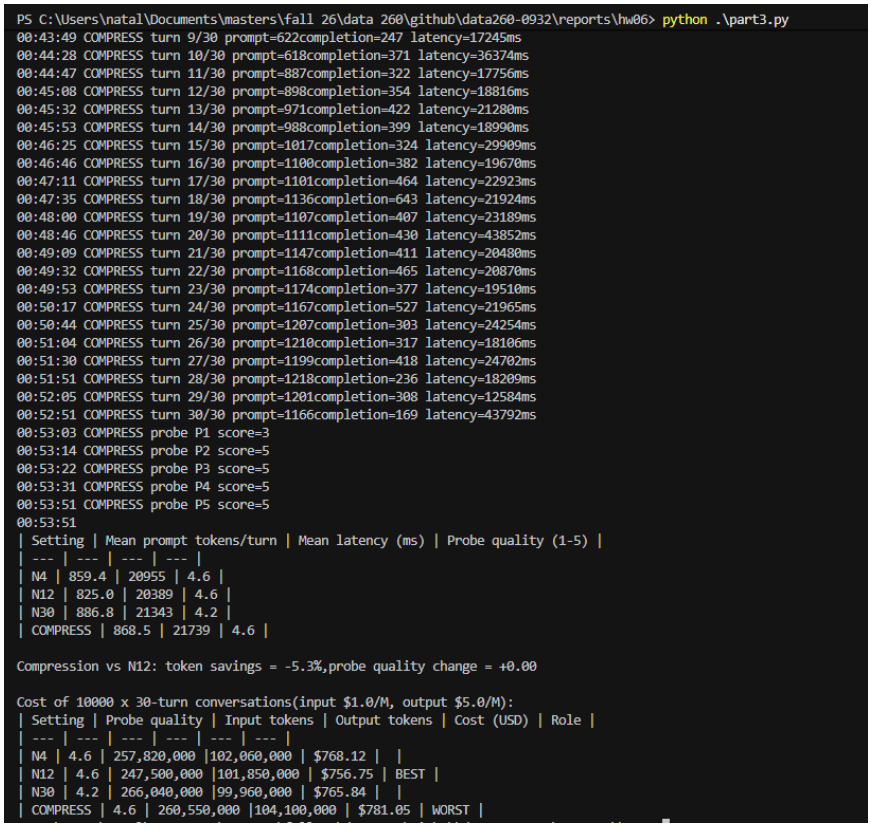
eviction/compression comparison results



```
PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw06> python .\part3.py
00:03:14  run N4: SHORT_TERM_N=4, policy=none
00:03:31 N4 turn 1/30 prompt=61completion=134 latency=14647ms
00:03:44 N4 turn 2/30 prompt=150completion=197 latency=11253ms
00:04:06 N4 turn 3/30 prompt=200completion=200 latency=20328ms
00:04:24 N4 turn 4/30 prompt=266completion=224 latency=15699ms
00:04:49 N4 turn 5/30 prompt=328completion=239 latency=22825ms
00:05:03 N4 turn 6/30 prompt=551completion=267 latency=12311ms
00:05:16 N4 turn 7/30 prompt=606completion=389 latency=11101ms
00:05:39 N4 turn 8/30 prompt=606completion=331 latency=20597ms
00:05:58 N4 turn 9/30 prompt=634completion=241 latency=17152ms
00:06:36 N4 turn 10/30 prompt=627completion=260 latency=35863ms
00:06:56 N4 turn 11/30 prompt=883completion=301 latency=17757ms
00:07:17 N4 turn 12/30 prompt=873completion=386 latency=19035ms
00:07:39 N4 turn 13/30 prompt=916completion=352 latency=20311ms
00:07:58 N4 turn 14/30 prompt=919completion=259 latency=16917ms
00:08:31 N4 turn 15/30 prompt=934completion=378 latency=31110ms
00:08:54 N4 turn 16/30 prompt=1019completion=507 latency=21159ms
00:09:20 N4 turn 17/30 prompt=1010completion=525 latency=23701ms
00:09:39 N4 turn 18/30 prompt=1075completion=325 latency=17227ms
00:10:04 N4 turn 19/30 prompt=1095completion=363 latency=22223ms
00:10:50 N4 turn 20/30 prompt=1118completion=321 latency=44154ms
00:11:12 N4 turn 21/30 prompt=1125completion=453 latency=20547ms
00:11:34 N4 turn 22/30 prompt=1151completion=397 latency=19956ms
00:11:55 N4 turn 23/30 prompt=1184completion=335 latency=18643ms
00:12:21 N4 turn 24/30 prompt=1178completion=625 latency=23184ms
00:12:49 N4 turn 25/30 prompt=1203completion=433 latency=26773ms
00:13:10 N4 turn 26/30 prompt=1197completion=396 latency=19099ms
00:13:35 N4 turn 27/30 prompt=1198completion=304 latency=22875ms
00:13:55 N4 turn 28/30 prompt=1190completion=214 latency=17747ms
00:14:12 N4 turn 29/30 prompt=1150completion=576 latency=14981ms
00:14:43 N4 turn 30/30 prompt=1335completion=273 latency=29285ms
00:14:56 N4 probe P1 score=3
00:15:06 N4 probe P2 score=5
00:15:15 N4 probe P3 score=5
00:15:27 N4 probe P4 score=5
00:15:44 N4 probe P5 score=5
00:15:44  run N12: SHORT_TERM_N=12, policy=none
00:15:58 N12 turn 1/30 prompt=61completion=111 latency=12071ms
00:16:11 N12 turn 2/30 prompt=132completion=219 latency=11191ms
00:16:34 N12 turn 3/30 prompt=177completion=247 latency=21105ms
00:16:51 N12 turn 4/30 prompt=237completion=156 latency=14758ms
00:17:15 N12 turn 5/30 prompt=298completion=204 latency=22148ms
00:17:29 N12 turn 6/30 prompt=535completion=242 latency=12287ms
00:17:47 N12 turn 7/30 prompt=588completion=686 latency=15135ms
00:18:11 N12 turn 8/30 prompt=578completion=348 latency=22152ms
00:18:33 N12 turn 9/30 prompt=617completion=447 latency=19920ms
00:19:03 N12 turn 10/30 prompt=612completion=369 latency=27821ms
00:19:22 N12 turn 11/30 prompt=794completion=266 latency=17008ms
00:19:44 N12 turn 12/30 prompt=804completion=454 latency=19930ms
00:20:06 N12 turn 13/30 prompt=856completion=327 latency=20775ms
00:20:27 N12 turn 14/30 prompt=904completion=316 latency=18087ms
00:20:56 N12 turn 15/30 prompt=894completion=248 latency=27335ms
00:21:17 N12 turn 16/30 prompt=966completion=358 latency=19361ms
00:21:41 N12 turn 17/30 prompt=987completion=322 latency=21203ms
00:22:01 N12 turn 18/30 prompt=1006completion=264 latency=16932ms
00:22:25 N12 turn 19/30 prompt=1002completion=382 latency=22569ms
00:23:08 N12 turn 20/30 prompt=1039completion=318 latency=40737ms
00:23:29 N12 turn 21/30 prompt=1089completion=356 latency=19230ms
```



```
PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw06> python .\part3.py
00:23:52 N12 turn 22/30 prompt=1111completion=405 latency=20172ms
00:24:12 N12 turn 23/30 prompt=1133completion=313 latency=18461ms
00:24:39 N12 turn 24/30 prompt=1141completion=730 latency=24549ms
00:25:04 N12 turn 25/30 prompt=1180completion=321 latency=23228ms
00:25:25 N12 turn 26/30 prompt=1195completion=418 latency=19151ms
00:25:52 N12 turn 27/30 prompt=1202completion=387 latency=24352ms
00:26:14 N12 turn 28/30 prompt=1216completion=392 latency=20413ms
00:26:31 N12 turn 29/30 prompt=1198completion=431 latency=14543ms
00:26:58 N12 turn 30/30 prompt=1197completion=147 latency=25032ms
00:27:11 N12 probe P1 score=3
00:27:23 N12 probe P2 score=5
00:27:34 N12 probe P3 score=5
00:27:45 N12 probe P4 score=5
00:28:07 N12 probe P5 score=5
00:28:07  run N30: SHORT_TERM_N=30, policy=none
00:28:22 N30 turn 1/30 prompt=61completion=111 latency=12143ms
00:28:36 N30 turn 2/30 prompt=132completion=219 latency=12177ms
00:28:59 N30 turn 3/30 prompt=177completion=248 latency=20903ms
00:29:17 N30 turn 4/30 prompt=238completion=299 latency=16683ms
00:29:41 N30 turn 5/30 prompt=331completion=216 latency=21900ms
00:29:55 N30 turn 6/30 prompt=527completion=209 latency=12029ms
00:30:06 N30 turn 7/30 prompt=588completion=207 latency=8920ms
00:30:29 N30 turn 8/30 prompt=594completion=275 latency=20168ms
00:30:50 N30 turn 9/30 prompt=633completion=371 latency=18956ms
00:31:29 N30 turn 10/30 prompt=627completion=356 latency=37338ms
00:31:51 N30 turn 11/30 prompt=875completion=343 latency=18189ms
00:32:12 N30 turn 12/30 prompt=900completion=377 latency=19363ms
00:32:34 N30 turn 13/30 prompt=957completion=302 latency=20089ms
00:32:56 N30 turn 14/30 prompt=980completion=284 latency=17558ms
00:33:27 N30 turn 15/30 prompt=985completion=349 latency=29314ms
00:33:49 N30 turn 16/30 prompt=1125completion=413 latency=20012ms
00:34:13 N30 turn 17/30 prompt=1152completion=368 latency=21696ms
00:34:33 N30 turn 18/30 prompt=1163completion=361 latency=18166ms
00:34:59 N30 turn 19/30 prompt=1171completion=438 latency=23647ms
00:35:56 N30 turn 20/30 prompt=1231completion=399 latency=55098ms
00:36:17 N30 turn 21/30 prompt=1227completion=365 latency=19511ms
00:36:40 N30 turn 22/30 prompt=1239completion=449 latency=20596ms
00:37:00 N30 turn 23/30 prompt=1242completion=290 latency=18162ms
00:37:23 N30 turn 24/30 prompt=1245completion=497 latency=21112ms
00:37:54 N30 turn 25/30 prompt=1266completion=420 latency=28070ms
00:38:14 N30 turn 26/30 prompt=1211completion=381 latency=18923ms
00:38:42 N30 turn 27/30 prompt=1210completion=492 latency=25521ms
00:39:02 N30 turn 28/30 prompt=1195completion=233 latency=18276ms
00:39:17 N30 turn 29/30 prompt=1160completion=435 latency=12762ms
00:39:52 N30 turn 30/30 prompt=1162completion=288 latency=33020ms
00:40:05 N30 probe P1 score=1
00:40:15 N30 probe P2 score=5
00:40:23 N30 probe P3 score=5
00:40:33 N30 probe P4 score=5
00:40:56 N30 probe P5 score=5
00:40:56  run COMPRESS: SHORT_TERM_N=12, policy=compress
00:41:14 COMPRESS turn 1/30 prompt=61completion=111 latency=14348ms
00:41:28 COMPRESS turn 2/30 prompt=132completion=223 latency=11594ms
00:41:51 COMPRESS turn 3/30 prompt=181completion=266 latency=21435ms
00:42:10 COMPRESS turn 4/30 prompt=244completion=271 latency=16516ms
00:42:38 COMPRESS turn 5/30 prompt=316completion=237 latency=26314ms
00:42:54 COMPRESS turn 6/30 prompt=529completion=380 latency=14126ms
00:43:06 COMPRESS turn 7/30 prompt=584completion=276 latency=9883ms
00:43:30 COMPRESS turn 8/30 prompt=595completion=349 latency=21559ms
```



```
PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw06> python .\part3.py
00:43:49 COMPRESS turn 9/30 prompt=622completion=247 latency=17245ms
00:44:28 COMPRESS turn 10/30 prompt=618completion=371 latency=36374ms
00:44:47 COMPRESS turn 11/30 prompt=887completion=322 latency=17756ms
00:45:08 COMPRESS turn 12/30 prompt=898completion=354 latency=18816ms
00:45:32 COMPRESS turn 13/30 prompt=971completion=422 latency=21280ms
00:45:53 COMPRESS turn 14/30 prompt=988completion=399 latency=18990ms
00:46:25 COMPRESS turn 15/30 prompt=1017completion=324 latency=29909ms
00:46:46 COMPRESS turn 16/30 prompt=1100completion=382 latency=19670ms
00:47:11 COMPRESS turn 17/30 prompt=1101completion=464 latency=22923ms
00:47:35 COMPRESS turn 18/30 prompt=1136completion=643 latency=21924ms
00:48:00 COMPRESS turn 19/30 prompt=1107completion=407 latency=23189ms
00:48:46 COMPRESS turn 20/30 prompt=1111completion=430 latency=43852ms
00:49:09 COMPRESS turn 21/30 prompt=1147completion=411 latency=20480ms
00:49:32 COMPRESS turn 22/30 prompt=1168completion=465 latency=20870ms
00:49:53 COMPRESS turn 23/30 prompt=1174completion=377 latency=19510ms
00:50:17 COMPRESS turn 24/30 prompt=1167completion=527 latency=21965ms
00:50:44 COMPRESS turn 25/30 prompt=1207completion=303 latency=24254ms
00:51:04 COMPRESS turn 26/30 prompt=1210completion=317 latency=18106ms
00:51:30 COMPRESS turn 27/30 prompt=1199completion=418 latency=24702ms
00:51:51 COMPRESS turn 28/30 prompt=1218completion=236 latency=18209ms
00:52:05 COMPRESS turn 29/30 prompt=1201completion=308 latency=12584ms
00:52:51 COMPRESS turn 30/30 prompt=1166completion=169 latency=43792ms
00:53:03 COMPRESS probe P1 score=3
00:53:14 COMPRESS probe P2 score=5
00:53:22 COMPRESS probe P3 score=5
00:53:31 COMPRESS probe P4 score=5
00:53:51 COMPRESS probe P5 score=5
00:53:51
| Setting | Mean prompt tokens/turn | Mean latency (ms) | Probe quality (1-5) |
| --- | --- | --- | --- |
| N4 | 859.4 | 20955 | 4.6 |
| N12 | 825.0 | 20389 | 4.6 |
| N30 | 886.8 | 21343 | 4.2 |
| COMPRESS | 868.5 | 21739 | 4.6 |

Compression vs N12: token savings = -5.3%,probe quality change = +0.00

Cost of 10000 x 30-turn conversations(input $1.0/M, output $5.0/M):
| Setting | Probe quality | Input tokens | Output tokens | Cost (USD) | Role |
| --- | --- | --- | --- | --- | --- |
| N4 | 4.6 | 257,820,000 |102,060,000 | $768.12 |  |
| N12 | 4.6 | 247,500,000 |101,850,000 | $756.75 | BEST |
| N30 | 4.2 | 266,040,000 |99,960,000 | $765.84 |  |
| COMPRESS | 4.6 | 260,550,000 |104,100,000 | $781.05 | WORST |
```

Part 4 - File-Based Memory and Consolidation

In this part, you will build a second memory system using AGENT.md and MEMORY.md,

then compare it with your MongoDB-based memory system.

1. Create the memory files

Reuse and expand the AGENT.md file from HW1.

• AGENT.md stores durable project and agent instructions.

AGENT.md > abc ## Prohibited actions and response rules

```
## Project description
- The project is an Open Source Vulnerability Management API with a React frontend.
- You are a code reviewer: the user pastes code and you point out the problems in it.

## Technology stack
- The backend uses Python 3, FastAPI, Pydantic, SQLAlchemy with MySQL, and PyMongo with MongoDB.
- The local language model runs through Ollama (qwen3:8b).

## Coding conventions
- Python names use snake_case and every function has a short comment.
- Route handlers must return correct HTTP status codes (400, 404, 422, 500).

## Testing approach
- Tests are written with pytest, and test names describe the behavior they check.

## Prohibited actions and response rules
You are a code reviewer. Follow the following rules/guidelines for every response:
- Answer only in bullet points
- Do not include any introductions, or summaries, or paragraphs
- Each bullet point will only show one issue
- Do not repeat bullet poinyd, do not repeat any code to user
- Do not show any code to user
- If there are no issues, reply with one bullet point telling user "There are no errors in the code"
```

• MEMORY.md stores durable facts learned from previous conversations.

MEMORY.md > abc # MEMORY

```
# MEMORY
- [user.name] The user's name is test (2026-10-18)
- [project.name] The project is the Open Source Vulnerability Management API (2026-10-18)
- [api.port] The API runs on port 8032 (2026-10-18)
```

2. Implement file-based memory

Create functions that:

• Load both Markdown files

```python
def load_files():
    return read_file(AGENT_FILE), read_file(MEMORY_FILE)

def is_comment(line):
    return line.strip().startswith("<!-- ")

def build_system_prompt():
    agent_text, memory_text = load_files()
    # the timestamp comment is only for the program, so leave it out of the prompt
    memory_for_prompt = "\n".join(l for l in memory_text.splitlines() if not is_comment(l))
    return agent_text + "\n\n# Long-term memory (from MEMORY.md)\n" + memory_for_prompt
```

```
def show_memory():
    agent_text, memory_text = load_files()
    needed, reason = needs_consolidation(memory_text)
    return ("AGENT.md\n" + agent_text +
        "\n\n MEMORY.md \n" + memory_text +
        f"\n\n[status] {count_memory_lines(memory_text)} memory lines | "
        f"consolidation needed: {needed} ({reason})")
```

• Add their contents to the system prompt

```
32  def build_system_prompt():
33      agent_text, memory_text = load_files()
34      # the timestamp comment is only for the program, so leave it out of the prompt
35      memory_for_prompt = "\n".join(l for l in memory_text.splitlines() if not is_comment(l))
36      return agent_text + "\n\n# Long-term memory (from MEMORY.md)\n" + memory_for_prompt
37
```

• Read the last-consolidated timestamp

```
38  def get_last_consolidated(text):
39      match = STAMP_PATTERN.search(text)
40      if not match:
41          return None
42      try:
43          when = datetime.fromisoformat(match.group(1))
44      except ValueError:
45          return None
46      if when.tzinfo is None:
47          when = when.replace(tzinfo=timezone.utc)
48      return when
49
```

• Determine when memory consolidation is required consolidate MEMORY.md when:

• It contains more than 40 non-empty, non-comment lines, or

• Its recorded consolidation time is more than 24 hours old

```
53  def count_memory_lines(text):
54      return sum(1 for line in text.splitlines() if line.strip() and not is_comment(line))
55
56  def needs_consolidation(text, now=None):
57      now = now or datetime.now(timezone.utc)
58      count = count_memory_lines(text)
59      if count > MAX_MEMORY_LINES:
60          return True, f"{count} non-empty, non-comment lines (limit {MAX_MEMORY_LINES})"
61      last = get_last_consolidated(text)
62      if last is None:
63          return True, "no last-consolidated timestamp is recorded"
64      age = now - last
65      if age > timedelta(hours=MAX_AGE_HOURS):
66          return True, f"last consolidated {age.total_seconds() / 3600 :.1f} hours ago (limit {MAX_AGE_HOURS})"
67      return False, f"{count} lines, consolidated {age.total_seconds() / 3600: .1f} hours ago"
68
```

3. Support memory commands

```
PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw06\backend> python -m app.file_mem_chat
[memory] consolidation needed: no last-consolidated timestamp is recorded -> type /compact
Paste code to review. Type /help for commands.
you> /help
Available commands:
/memory show the current file-based memory (AGENT.md and MEMORY.md)
/ compact consolidate MEMORY.md and compress the chat history; shows counts before and after
/dream find useful long-term facts in the recent conversation and add them to MEMORY.md
/help show this list
/exit quit
you> /memory
AGENT.md
## Project description
- The project is an Open Source Vulnerability Management API with a React frontend.
- You are a code reviewer: the user pastes code and you point out the problems in it.

## Technology stack
- The backend uses Python 3, FastAPI, Pydantic, SQLAlchemy with MySQL, and PyMongo with MongoDB.
- The local language model runs through Ollama (qwen3:8b).

## Coding conventions
- Python names use snake_case and every function has a short comment.
- Route handlers must return correct HTTP status codes (400, 404, 422, 500).

## Testing approach
- Tests are written with pytest, and test names describe the behavior they check.

## Prohibited actions and response rules
You are a code reviewer. Follow the following rules/guidelines for every response:
- Answer only in bullet points
- Do not include any introductions, or summaries, or paragraphs
- Each bullet point will only show one issue
- Do not repeat bullet poinyd, do not repeat any code to user
- Do not show any code to user
- If there are no issues, reply with one bullet point telling user "There are no errors in the code"

 MEMORY.md
# MEMORY
- [user.name] The user's name is test (2026-10-18)
- [project.name] The project is the Open Source Vulnerability Management API (2026-10-18)
- [api.port] The API runs on port 8032 (2026-10-18)

[status] 4 memory lines | consolidation needed: True (no last-consolidated timestamp is recorded)
you> /compact
[/compact] MEMORY.md lines: 4 -> 4(duplicates merged: 0, contradictions resolved: 0)
[/compact] Conversation messages: 1 -> 1
you> /dream
[/dream] facts added to MEMORY.md: 0
you> /exit
```

Implement the following commands:

• /memory — display the current file-based memory

```python
if user_input == "/memory":
    print(fm.show_memory())
    continue
```

```python
def show_memory():
    agent_text, memory_text = load_files()
    needed, reason = needs_consolidation(memory_text)
    return ("AGENT.md\n" + agent_text +
        "\n\n MEMORY.md \n" + memory_text +
        f"\n\n[status] {count_memory_lines(memory_text)} memory lines | "
        f"consolidation needed: {needed} ({reason})")
```

• /compact — consolidate MEMORY.md and report the message count before and After

```python
if user_input == "/compact":
    messages, text = fm.run_compact (messages)
    print(text)
    continue
```

```python
def run_compact(messages) :
    report = consolidate_file()
    before_msgs = len(messages)
    new_messages = compact_history(messages)
    new_messages[0] = {"role": "system", "content": build_system_prompt()}
    text = (f"[/compact] MEMORY.md lines: {report['lines_before']} -> {report['lines_after']}"
        f"(duplicates merged: {len(report['duplicates'])}, "
        f"contradictions resolved: {len(report['resolutions'])})\n"
        f"[/compact] Conversation messages: {before_msgs} -> {len(new_messages)}")
    for r in report["suspect_resolutions"]:
        text += f"\n[/compact] CHECK [{r['topic']}]: newest-wins may be wrong, see {os.path.basename(ARCHIVE_FILE)}"
    return new_messages, text
```

• /dream — identify useful long-term facts from recent conversation and add them to MEMORY.md

```python
if user_input == "/dream":
    added = fm.run_dream(messages [-10: ])
    print(f"[/dream] facts added to MEMORY.md: {len(added)}")
    for line in added:
        print(" ", line)
    messages[0] = {"role": "system", "content": fm.build_system_prompt()}
    needed, reason = fm.needs_consolidation(fm.read_file(fm.MEMORY_FILE))
    if needed:
        print("[memory] consolidation needed:", reason, "-> type /compact")
    continue
```

```python
def run_dream(recent_messages):
    from . import llm_adapter
    transcript = "\n".join(m["role"] +": "+ m["content"]
    for m in recent_messages if m["role"] != "system")
    if not transcript.strip():
        return []
    prompt = ("Read the conversation below. List up to 5 long-term facts worth remembering "
        "(user details, project decisions, preferences). Reply with ONLY a JSON list like "
        '[{"topic": "user.name", "fact": "The user is named test"}].'
        "topic is a short lowercase label with dots. If there is nothing, reply [].\n\n"
        + transcript)
    reply, _, _= llm_adapter.chat([{"role": "user", "content": prompt}])
    start, end = reply.find("["), reply.rfind("]")
    if start == -1 or end == -1:
        return []
    try:
        items = json.loads(reply[start:end + 1])
    except json.JSONDecodeError :
        return []

    existing,_ = parse_memory(read_file(MEMORY_FILE))
    seen = {(f["topic"].lower(), normalize(f["text"])) for f in existing}
    today = datetime.now(timezone.utc).strftime("%Y-%m-%d")
    added = []
    for item in items[:5]:
        if not isinstance(item, dict) or not item.get("topic") or not item.get("fact"):
            continue
        topic = str(item["topic"]).strip()
        fact = str(item["fact"]).strip()
        if (topic.lower(), normalize(fact)) in seen:
            continue
        added.append(f"- [{topic}] {fact} ({today})")
    if added:
        text = read_file(MEMORY_FILE)
        if text and not text.endswith("\n"):
            text += "\n"
        write_file(MEMORY_FILE, text + "\n". join(added) + "\n")
    return added
```

• /help — explain the available commands

```python
if user_input == "/help":
    print(fm.help_text())
    continue
```

```python
def help_text():
    return ("Available commands: \n"
        "/memory show the current file-based memory (AGENT.md and MEMORY.md) \n"
        "/ compact consolidate MEMORY.md and compress the chat history; shows counts before and after\n"
        "/dream find useful long-term facts in the recent conversation and add them to MEMORY.md\n"
        "/help show this list\n"
        "/exit quit")
```

4. Expand and test AGENT.md

By the end of HW6, AGENT.md must contain at least five instructions covering at least three of these categories:

• Project description
• Technology stack
• Coding conventions
• Testing approach
• Prohibited actions

AGENT.md > abc ## Prohibited actions and response rules

```
## Project description
- The project is an Open Source Vulnerability Management API with a React frontend.
- You are a code reviewer: the user pastes code and you point out the problems in it.

## Technology stack
- The backend uses Python 3, FastAPI, Pydantic, SQLAlchemy with MySQL, and PyMongo with MongoDB.
- The local language model runs through Ollama (qwen3:8b).

## Coding conventions
- Python names use snake_case and every function has a short comment.
- Route handlers must return correct HTTP status codes (400, 404, 422, 500).

## Testing approach
- Tests are written with pytest, and test names describe the behavior they check.

## Prohibited actions and response rules
You are a code reviewer. Follow the following rules/guidelines for every response:
- Answer only in bullet points
- Do not include any introductions, or summaries, or paragraphs
- Each bullet point will only show one issue
- Do not repeat bullet poinyd, do not repeat any code to user
- Do not show any code to user
- If there are no issues, reply with one bullet point telling user "There are no errors in the code"
```

Generate or implement one function and verify that it follows the instructions in AGENT.md.

```python
def review_code(code_text):
    from . import llm_adapter
    messages = [{"role": "system", "content": build_system_prompt()},
        {"role": "user", "content": code_text}]
    reply, _, _= llm_adapter.chat(messages)
    return reply

INTRO_WORDS = ["here are", "here is", "sure", "certainly", "i found", "below", "let me", "the following"]
SUMMARY_WORDS = ["in summary", "summary", "overall", "in conclusion", "to summarize", "to sum up"]

def check_reply_follows_agent_md(reply, expect_no_issue=False):
    lines = [l.strip() for l in reply.splitlines() if l.strip()]
    bullets = [l[2:].strip()
               if l[:2] in ("- ", "*","•")
               else None for l in lines]
    results = []
    bad = [l for l, b in zip(lines, bullets) if b is None]
    results.append(("only bullet points", len(lines) > 0 and not bad,
        "all lines are bullets" if not bad else "not a bullet: " + bad[0][:40]))

    texts = [b for b in bullets if b is not None]
    starts = [t.lower() for t in texts]
    intro = any(s.startswith(w) for s in starts[:1] for w in INTRO_WORDS)
    summary = any(s.startswith(w) for s in starts[-1:] for w in SUMMARY_WORDS)
    results.append(("no introduction or summary", not intro and not summary,
        "none found" if not (intro or summary) else "intro or summary bullet found"))
    too_long = [t for t in texts
                if len(t.split()) > 35 or ". " in t]
    results.append(("one issue per bullet", not too_long,
        "every bullet is short and one sentence" if not too_long else "long bullet: " + too_long[0][:40]))
    normal = [normalize(t) for t in texts]
    results.append(("no repeated bullets", len(normal) == len(set(normal)),
        "all bullets are different" if len(normal) == len(set(normal)) else "a bullet is repeated"))
    has_code = "```" in reply or "`" in reply
    results.append(("no code shown", not has_code, "no backticks found" if not has_code else "backticks found"))
    if expect_no_issue:
        ok = len(texts) == 1 and normalize(texts[0]) == normalize("There are no errors in the code")
        results.append(("no-issue reply", ok,
            "exact sentence used" if ok else "expected: There are no errors in the code"))
    return results

```

```
PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw06\backend> python .\test_agent.py

  === case: buggy code  ===
reply:
- The function does not handle the case where the input list is empty, leading to a ZeroDivisionError
- The except block is too broad and silently ignores errors instead of raising exceptions
- No input validation is performed for non-list types (e.g. None or integers)
- The function returns None on failure instead of raising appropriate exceptions
- The code lacks documentation for expected input/output behavior
  [PASS] only bullet points: all lines are bullets
  [PASS] no introduction or summary: none found
  [PASS] one issue per bullet: every bullet is short and one sentence
  [PASS] no repeated bullets: all bullets are different
  [PASS] no code shown: no backticks found

  === case: clean code  ===
reply:
- There are no errors in the code
  [PASS] only bullet points: all lines are bullets
  [PASS] no introduction or summary: none found
  [PASS] one issue per bullet: every bullet is short and one sentence
  [PASS] no repeated bullets: all bullets are different
  [PASS] no code shown: no backticks found
  [PASS] no-issue reply: exact sentence used

OVERALL: ALL RULES FOLLOWED
```

5. Test consolidation

Create a 55-line MEMORY.md containing:

• At least three duplicated facts

• At least three deliberately contradictory facts

reports > hw06 > testing > MEMORY_original.md > ...

```
<!-- last_consolidated: 2026-10-01T08:00:00+00:00 -->
# MEMORY

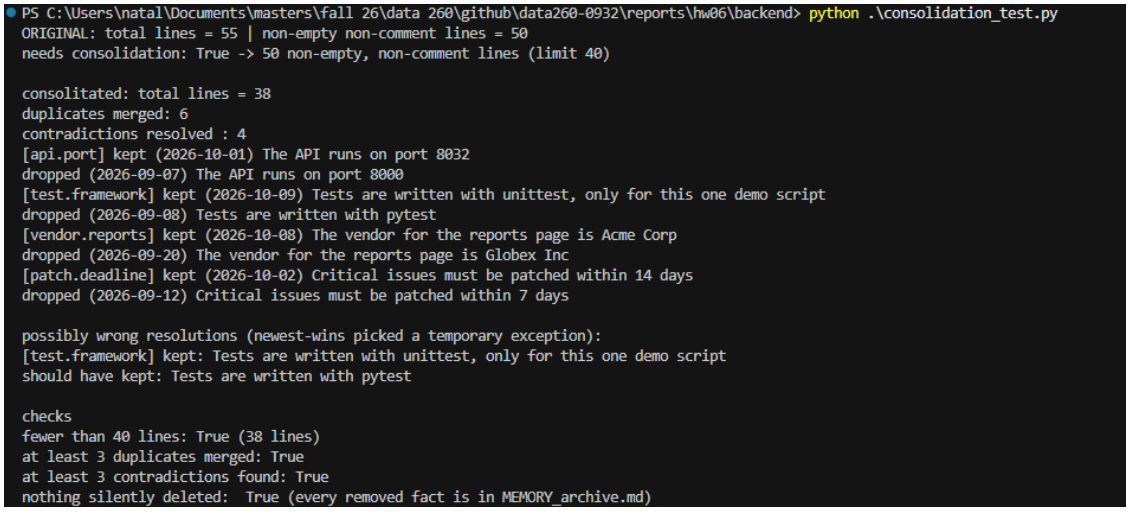
## User
- [user.name] The user's name is Alice (2026-09-01)
- [user.employer] The user works at Northwind Bank (2026-09-01)
- [user.role] The user is a security analyst (2026-09-01)
- [user.name] the user's name is alice. (2026-09-03)
- [user.timezone] The user works in the US Pacific time zone (2026-09-02)
- [user.experience] The user has three years of Python experience (2026-09-02)
- [user.employer] The user works at Northwind Bank. (2026-09-10)
- [user.format] The user prefers answers in bullet points (2026-09-04)
- [user.learning] The user is learning MongoDB this semester (2026-09-05)
- [user.course] The user is enrolled in DATA-260 at SJSU (2026-09-05)

## Project
- [project.name] The project is the Open Source Vulnerability Management API (2026-09-01)
- [stack.backend] The backend uses Python and FastAPI (2026-09-01)
- [stack.frontend] The frontend uses React with Redux (2026-09-01)
- [stack.nosql] MongoDB stores the document and memory collections (2026-09-06)
- [db.relational] The relational data is stored in MySQL through SQLAlchemy (2026-09-02)
- [api.port] The API runs on port 8000 (2026-09-07)
- [db.name] The MongoDB database is called s0932_doc (2026-09-08)
- [project.prefix] The student prefix is s0932 (2026-09-08)
- [project.seed] The seed value is 932 (2026-09-08)
- [project.name] The project is the Open Source Vulnerability Management API. (2026-09-15)
- [stack.backend] the backend uses python and fastapi (2026-09-15)
- [stack.frontend] The frontend uses React with Redux. (2026-09-15)
- [stack.nosql] MongoDB stores the document and memory collections. (2026-09-16)
- [api.port] The API runs on port 8032 (2026-10-01)
- [embeddings.model] Embeddings are created with nomic-embed-text (2026-09-20)

## Preferences
- [11m.model] The local model is qwen3:8b served by Ollama (2026-09-09)
- [test.framework] Tests are written with pytest (2026-09-08)
- [code.style] Python names use snake_case (2026-09-08)
- [code.comments] Every function has a short comment (2026-09-08)
- [review.format] Code reviews are given as bullet points only (2026-09-09)
- [memory.threshold] MEMORY.md is consolidated when it passes 40 lines (2026-09-10)
- [llm.model] The local model is qwen3:8b served by Ollama. (2026-09-18)
- [test.framework] Tests are written with unittest, only for this one demo script (2026-10-09)
- [api.docs] The Swagger page is at /docs (2026-09-11)
- [cors.origin] CORS allows the React dev server on localhost:5173 (2026-09-11)
- [session.ttl] Login sessions expire after 30 minutes (2026-09-12)

## Team
- [patch.deadline] Critical issues must be patched within 7 days (2026-09-12)
- [vendor.reports] The vendor for the reports page is Acme Corp (2026-10-08)
- [owner.login] The login service is owned by Dolittle (2026-09-13)
- [owner.reports] The reports page is owned by Persona B (2026-09-13)
- [team.size] The security team has six people (2026-09-14)
- [team.standup] The team stand-up is every weekday at 9:30 (2026-09-14)
- [vendor.reports] The vendor for the reports page is Globex Inc (2026-09-20)
- [patch.deadline] Critical issues must be patched within 14 days (2026-10-02)
- [repo.tag] The submission tag is hw6 (2026-10-15)
```

Run your consolidation process and reduce the file to fewer than 40 lines.



Your consolidation must:

- Merge duplicate facts
- Resolve contradictions explicitly

```python
def consolidate_text(text, now=None):
    now = now or datetime.now(timezone.utc)
    facts, notes = parse_memory(text)

    kept = {}
    order = []
    duplicates = []
    resolutions = []

    for fact in facts:
        key = fact["topic"].lower()
        if key not in kept:
            kept [key] = fact
            order.append(key)
            continue
        old = kept[key]
        if normalize(old["text"]) == normalize(fact["text"]):
            # keep only one of the dumplicates
            newest_date = max(old["date"], fact["date"])
            kept [key] = {"topic": old["topic"], "text": old["text"], "date": newest_date}
            duplicates.append({"topic": fact["topic"], "dropped": fact, "kept": kept[key]})
        else:
            if fact["date"] >= old["date"]:
                winner, loser = fact, old
            else:
                winner, loser = old, fact
            kept [key ] = winner
            resolutions.append({"topic": winner["topic"], "kept": winner, "dropped": loser,
            "suspect": looks_temporary(winner["text"])})

    lines = [make_stamp_line(now), "# MEMORY", ""]
    for key in order:
        f = kept [key ]
        lines.append(f"- [{f[ 'topic']}] {f['text']} ({f['date']})")
    if notes:
        lines += ["", "## Other notes"] + notes
    new_text = "\n".join(lines) + "\n"
    archive_lines = []
    for d in duplicates:
        x = d["dropped"]
        archive_lines.append(f'- MERGED_DUPLICATE [{d["topic"]}] "{x["text"]}" ({x["date"]}) '
            f' | same as kept fact dated {d["kept"]["date"]}')
    for r in resolutions:
        k, x = r["kept"],r["dropped"]
        line = (f'- SUPERSEDED [{r["topic"]}] "{x["text"]}" ({x["date"]})'
        f'| replaced by "{k["text"]}" ({k["date"]}) | rule: newest date wins')
        if r["suspect"]:
            line += " | CHECK: newer fact sounds temporary, this may be the WRONG choice"
        archive_lines.append(line)
    if archive_lines:
        archive_lines.insert(0, " <!-- consolidation at " + now.isoformat() + " -->")

    report = {"facts_before": len(facts), "facts_after": len(kept),
        "duplicates": duplicates, "resolutions": resolutions,
        "suspect_resolutions": [r for r in resolutions if r["suspect"]],
        "archive_lines": archive_lines}
    return new_text, report
```

- Avoid silently deleting conflicting information

```python
def consolidate_file(memory_path=MEMORY_FILE, archive_path=ARCHIVE_FILE):
    old_text = read_file(memory_path)
    new_text, report = consolidate_text(old_text)
    if report["archive_lines"]:
        with open(archive_path, "a", encoding="utf-8") as f:
            f.write("\n".join(report["archive_lines"]) + "\n")
    write_file(memory_path, new_text)
    report["lines_before"] = count_memory_lines(old_text)
    report["lines_after"] = count_memory_lines(new_text)
    report["total_lines_before"] = len(old_text.splitlines())
    report["total_lines_after"] = len(new_text.splitlines())
    return report
```

reports > hw06 > testing > MEMORY_archive.md

```
<!-- consolidation at 2026-10-19T08:44:03.176959+00:00 -->
- MERGED_DUPLICATE [user.name] "the user's name is alice." (2026-09-03)  | same as kept fact dated 2026-09-03
- MERGED_DUPLICATE [user.employer] "The user works at Northwind Bank." (2026-09-10)  | same as kept fact dated 2026-09-10
- MERGED_DUPLICATE [project.name] "The project is the Open Source Vulnerability Management API." (2026-09-15)  | same as kept fact dated 2026-09-15
- MERGED_DUPLICATE [stack.backend] "the backend uses python and fastapi" (2026-09-15)  | same as kept fact dated 2026-09-15
- MERGED_DUPLICATE [stack.frontend] "The frontend uses React with Redux." (2026-09-15)  | same as kept fact dated 2026-09-15
- MERGED_DUPLICATE [stack.nosql] "MongoDB stores the document and memory collections." (2026-09-16)  | same as kept fact dated 2026-09-16
- SUPERSEDED [api.port] "The API runs on port 8000" (2026-09-07)| replaced by "The API runs on port 8032" (2026-10-01) | rule: newest date wins
- SUPERSEDED [test.framework] "Tests are written with pytest" (2026-09-08)| replaced by "Tests are written with unittest, only for this one demo script" (2026-10-09) | rule: newest date wins | CHECK: newer fact sounds temporary, this may be the WRONG choice
- SUPERSEDED [vendor.reports] "The vendor for the reports page is Globex Inc" (2026-09-20)| replaced by "The vendor for the reports page is Acme Corp" (2026-10-08) | rule: newest date wins
- SUPERSEDED [patch.deadline] "Critical issues must be patched within 7 days" (2026-09-12)| replaced by "Critical issues must be patched within 14 days" (2026-10-02) | rule: newest date wins

```

• Identify any contradiction that was resolved incorrectly

```python
RISKY_WORDS = ["only", "for now", "temporar", "just for", "this week", "today", "one-time", "this one"]
```

```python
def looks_temporary(text):
    lowered = text.lower ()
    return any(word in lowered for word in RISKY_WORDS)
```

```python
    for fact in facts:
        key = fact["topic"].lower()
        if key not in kept:
            kept [key] = fact
            order.append(key)
            continue
        old = kept[key]
        if normalize(old["text"]) == normalize(fact["text"]):
            # keep only one of the dumplicates
            newest_date = max(old["date"], fact["date"])
            kept [key] = {"topic": old["topic"], "text": old["text"], "date": newest_date}
            duplicates.append({"topic": fact["topic"], "dropped": fact, "kept": kept[key]})
        else:
            if fact["date"] >= old["date"]:
                winner, loser = fact, old
            else:
                winner, loser = old, fact
            kept [key ] = winner
            resolutions.append({"topic": winner["topic"], "kept": winner, "dropped": loser,
            "suspect": looks_temporary(winner["text"])})

    lines = [make_stamp_line(now), "# MEMORY", ""]
    for key in order:
        f = kept [key ]
        lines.append(f"- [{f[ 'topic']}] {f['text']} ({f['date']})")
    if notes:
        lines += ["", "## Other notes"] + notes
    new_text = "\n".join(lines) + "\n"
    archive_lines = []
    for d in duplicates:
        x = d["dropped"]
        archive_lines.append(f'- MERGED_DUPLICATE [{d["topic"]}] "{x["text"]}" ({x["date"]}) '
            f' | same as kept fact dated {d["kept"]["date"]}')
    for r in resolutions:
        k, x = r["kept"],r["dropped"]
        line = (f'- SUPERSEDED [{r["topic"]}] "{x["text"]}" ({x["date"]})'
        f'| replaced by "{k["text"]}" ({k["date"]}) | rule: newest date wins')
        if r["suspect"]:
            line += " | CHECK: newer fact sounds temporary, this may be the WRONG choice"
        archive_lines.append(line)
    if archive_lines:
        archive_lines.insert(0, " <!-- consolidation at " + now.isoformat() + " -->")

    report = {"facts_before": len(facts), "facts_after": len(kept),
        "duplicates": duplicates, "resolutions": resolutions,
        "suspect_resolutions": [r for r in resolutions if r["suspect"]],
        "archive_lines": archive_lines}
    return new_text, report
```

reports > hw06 > testing > MEMORY_archive.md

```
<!-- consolidation at 2026-10-19T08:44:03.176959+00:00 -->
- MERGED_DUPLICATE [user.name] "the user's name is alice." (2026-09-03)  | same as kept fact dated 2026-09-03
- MERGED_DUPLICATE [user.employer] "The user works at Northwind Bank." (2026-09-10)  | same as kept fact dated 2026-09-10
- MERGED_DUPLICATE [project.name] "The project is the Open Source Vulnerability Management API." (2026-09-15)  | same as kept fact dated 2026-09-15
- MERGED_DUPLICATE [stack.backend] "the backend uses python and fastapi" (2026-09-15)  | same as kept fact dated 2026-09-15
- MERGED_DUPLICATE [stack.frontend] "The frontend uses React with Redux." (2026-09-15)  | same as kept fact dated 2026-09-15
- MERGED_DUPLICATE [stack.nosql] "MongoDB stores the document and memory collections." (2026-09-16)  | same as kept fact dated 2026-09-16
- SUPERSEDED [api.port] "The API runs on port 8000" (2026-09-07)| replaced by "The API runs on port 8032" (2026-10-01) | rule: newest date wins
- SUPERSEDED [test.framework] "Tests are written with pytest" (2026-09-08)| replaced by "Tests are written with unittest, only for this one demo script" (2026-10-09) | rule: newest date wins | CHECK: newer fact sounds temporary, this may be the WRONG choice
- SUPERSEDED [vendor.reports] "The vendor for the reports page is Globex Inc" (2026-09-20)| replaced by "The vendor for the reports page is Acme Corp" (2026-10-08) | rule: newest date wins
- SUPERSEDED [patch.deadline] "Critical issues must be patched within 7 days" (2026-09-12)| replaced by "Critical issues must be patched within 14 days" (2026-10-02) | rule: newest date wins

```

Commit both the original and consolidated versions to Git.

MEMORY.md > abc # MEMORY

```
# MEMORY
- [user.name] The user's name is test (2026-10-18)
- [project.name] The project is the Open Source Vulnerability Management API (2026-10-18)
- [api.port] The API runs on port 8032 (2026-10-18)
```

reports > hw06 > testing > MEMORY_consolidated.md

```
<!-- last_consolidated: 2026-10-19T08:44:03.176959+00:00 -- >
# MEMORY

- [user.name] The user's name is Alice (2026-09-03)
- [user.employer] The user works at Northwind Bank (2026-09-10)
- [user.role] The user is a security analyst (2026-09-01)
- [user.timezone] The user works in the US Pacific time zone (2026-09-02)
- [user.experience] The user has three years of Python experience (2026-09-02)
- [user.format] The user prefers answers in bullet points (2026-09-04)
- [user.learning] The user is learning MongoDB this semester (2026-09-05)
- [user.course] The user is enrolled in DATA-260 at SJSU (2026-09-05)
- [project.name] The project is the Open Source Vulnerability Management API (2026-09-15)
- [stack.backend] The backend uses Python and FastAPI (2026-09-15)
- [stack.frontend] The frontend uses React with Redux (2026-09-15)
- [stack.nosql] MongoDB stores the document and memory collections (2026-09-16)
- [db.relational] The relational data is stored in MySQL through SQLAlchemy (2026-09-02)
- [api.port] The API runs on port 8032 (2026-10-01)
- [db.name] The MongoDB database is called s0932_doc (2026-09-08)
- [project.prefix] The student prefix is s0932 (2026-09-08)
- [project.seed] The seed value is 932 (2026-09-08)
- [embeddings.model] Embeddings are created with nomic-embed-text (2026-09-20)
- [llm.model] The local model is qwen3:8b served by Ollama (2026-09-09)
- [test.framework] Tests are written with unittest, only for this one demo script (2026-10-09)
- [code.style] Python names use snake_case (2026-09-08)
- [code.comments] Every function has a short comment (2026-09-08)
- [review.format] Code reviews are given as bullet points only (2026-09-09)
- [memory.threshold] MEMORY.md is consolidated when it passes 40 lines (2026-09-10)
- [llm.model] The local model is qwen3:8b served by Ollama. (2026-09-18)
- [api.docs] The Swagger page is at /docs (2026-09-11)
- [cors.origin] CORS allows the React dev server on localhost:5173 (2026-09-11)
- [session.ttl] Login sessions expire after 30 minutes (2026-09-12)
- [patch.deadline] Critical issues must be patched within 14 days (2026-10-02)
- [vendor.reports] The vendor for the reports page is Acme Corp (2026-10-08)
- [owner.login] The login service is owned by Dolittle (2026-09-13)
- [owner.reports] The reports page is owned by Persona B (2026-09-13)
- [team.size] The security team has six people (2026-09-14)
- [team.standup] The team stand-up is every weekday at 9:30 (2026-09-14)
- [repo.tag] The submission tag is hw6 (2026-10-15)
```

6. Compare both memory systems

Compare MongoDB-backed memory and file-based memory in terms of:

• Inspectability

• Persistence after a restart

• Scalability across many sessions

• Suitability for facts that must not be forgotten

The MongoDB memory needs the Compass program to be able to read the files because of the embeddings. The file based memory on the other hand as the md files for the agent and memory that you can edit and is human readable. The MongoDB persists in the database volume and tile memory persists in plain files. If the files aren't saved, it won't persist. For scalability, MongoDB is good because it has indexes and can keep track of various user sessions, vector retrieval. File memory is not that good at this because every fact needs a prompt, so it can be quite costly when the file size dramatically increases. MongoDB memory can miss facts that must not be forgotten

because some important information can be lost when it summarizes. File memory is better at this because all the facts are visible and an import fact cannot be skipped.

AI_USE.md

reports > hw06 > AI_USE.md

```
1. I used ai assistance to help me figure out file memory for the homework because I am unfamilar with it. I needed it to help understand how to test the memory in order to get the memory after consolidation and before. I also used it to help me draft what to ask at ollama model for testing.
2. AI produced the wrong information when helping me pull from the agent and memory md files. It was not pulling from the right location and writing correctly/ overwritting what I wanted. So this was an issue I had to fix by debugging my code how it called and why it was overwritting with the wrong information.
3. I detected the problem above by checking the file directory locations. I had wrote to the wrong location and so the pulling was correct. I also made different files for the memory after consolidation so it did not overwrite my file and cause errors when I was running the consolidation test file. I verified that the consolidation was removing any duplicates and everything based on the assignment details manually and made sure the file afterwards was 55 lines as mentioned.
4. I changed the file directory it was writing to and pulling from and also created different files to handle the memory after consolidation. Previously, it was getting the files incorrectly at wrong locations and overwritting the file. I consulted the homework file to determine that there should be two memory files and this also solved my errors with it not getting the duplicates correctly and also the 55 lines output I wanted.
```

Verification

```
PS C:\Users\natal\Documents\masters\fall 26\data 260\github\data260-0932\reports\hw06> python .\verify_hw06.py
PASS - backend responds on PORT_BASE - GET /health returned 200
PASS - document API create, read, delete works - create 201, get 200, delete 200, get after delete 404
PASS - invalid document is rejected with 422 - POST with an invalid category returned 422
PASS - POST /api/chat returns a reply - POST /api/chat returned 200, reply length: 29
PASS - user and assistant messages are saved - GET /api/memory returned 2 messages, user + assistant saved: True
PASS - short-term window is used on the next turn - second turn short_term_count = 2 (needs 2 or more)
PASS - aggregate endpoint counts messages - GET /api/aggregate returned 200, total messages counted: 4
PASS - memory is isolated between users - a different user_id sees no messages or facts: True
PASS - chat request is logged with tokens and latency - chat_usage.csv rows 144 -> 146, token and latency columns present: True
PASS - AGENT.md and MEMORY.md load into the system prompt - AGENT.md chars: 1101, MEMORY.md chars: 264, AGENT.md in system prompt: True
PASS - consolidation trigger (over 40 lines / 24 hours) - 41 lines needs consolidation: True, small recent file needs it: False
verification.json outputted
```